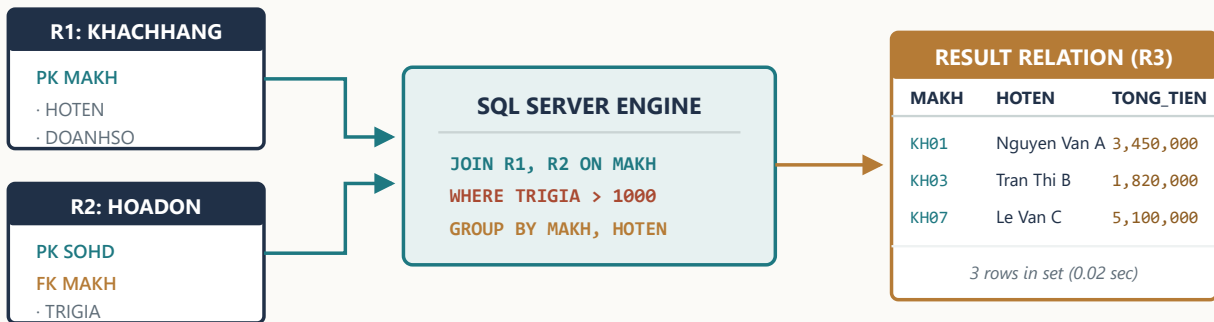


IT004

# THỰC HÀNH CƠ SỞ DỮ LIỆU

BIÊN SOẠN: VÕ TRỌNG PHÚC



## Thông tin biên soạn

**Biên soạn và hệ thống hóa:** Võ Trọng Phúc

**Môn học:** IT004 — Cơ sở dữ liệu (Phần Thực hành T-SQL & SQL Server)

**Đối tượng phục vụ:** Sinh viên theo học chương trình Cử nhân Công nghệ Thông tin, Hệ thống Thông tin, Kỹ thuật Phần mềm và các ngành kỹ thuật liên quan.

## Mục đích & Phương pháp tiếp cận

Cẩm nang Thực hành là ấn phẩm đồng hành cùng *Sổ tay lý thuyết IT004*, tập trung chuyển hóa các mô hình toán học (Đại số quan hệ, Ràng buộc toàn vẹn, Phụ thuộc hàm) thành mã thực thi Transact-SQL (T-SQL) trên hệ quản trị cơ sở dữ liệu Microsoft SQL Server.

Tài liệu xây dựng phương pháp tiếp cận dựa trên 3 trụ cột kỹ thuật:

- **Kiểm tra có thể tái lập:** Mỗi câu lệnh gắn với lược đồ đầu vào, TRACE SUY LUẬN LOGIC và bảng kết quả có nhãn STATIC, RUNTIME hoặc ILLUSTRATIVE.
- **Tư duy dạng tập hợp (Set-Based Reasoning):** Triệt để giải thích và loại bỏ các thói quen lập trình duyệt con trỏ/vô hướng không tối ưu trên SQL Server.
- **Kỹ thuật Trigger an toàn đa dòng (Multi-Row Safety):** Thiết kế các giải pháp ràng buộc dữ liệu đảm bảo toàn vẹn ngay cả khi thực hiện thao tác DML hàng loạt.

## Căn cứ học thuật & Tiêu chuẩn

Nội dung cẩm nang được xây dựng và đối chiếu chuẩn xác với:

- Tài liệu kỹ thuật chính thức **Microsoft Learn T-SQL Reference** (các mã nguồn định danh TECH-A01 đến TECH-A11).
- Bộ bài kiểm chứng khảo sát **Phase A:** gồm 17 đề thi tự luận/trắc nghiệm, 10 canonical practical-exam artifacts trong snapshot đóng băng và tiến trình 4 bài thực hành Lab 01–04.

### Lưu ý về phạm vi và bản quyền

Mọi ví dụ, câu hỏi và lời giải trong tài liệu được biên soạn độc lập, chuẩn hóa cú pháp và bổ sung phân tích cơ chế thực thi chi tiết. Tài liệu không sao chép nguyên văn các bản giải thô của sinh viên.

# Kiến trúc Cẩm nang Thực hành

Kiến trúc sản xuất gồm Phần 0–6, Labs 01–04, debugging, quy trình thi và phụ lục. Số trang không in để tránh stale pagination.

## Phần 0: Môi trường & Quy trình SSMS

- 0.1 Cấu trúc SQL Server & SSMS
- 0.2 Khối lệnh GO & Phạm vi biến
- 0.3 Checklist 60 giây trước Execute
- 0.4 Cô lập giao dịch kiểm thử

## Phần 1: Xây dựng CSDL với DDL

- 1.1 CREATE TABLE, PK, FK
- 1.2 CHECK, DEFAULT, Kiểu dữ liệu
- 1.3 Tràn số & Bẫy Unicode
- 1.4 Thứ tự chạy Script DDL

## Phần 2: Thao tác Dữ liệu với DML

- 2.1 INSERT đơn & đa dòng
- 2.2 UPDATE có tính toán & Subquery
- 2.3 DELETE an toàn & Khóa ngoại
- 2.4 Quy trình kiểm chứng 3 bước

## Phần 3: Truy vấn Nền tảng & NULL

- 3.1 SELECT, WHERE, LIKE, BETWEEN
- 3.2 Logic 3 giá trị & Bẫy IS NULL
- 3.3 Sắp xếp kết quả ORDER BY

## Phần 4: Truy vấn Kết nối & Gom nhóm

- 4.1 Phép kết INNER & OUTER JOIN
- 4.2 Xử lý truy vấn logic (Logical Query Processing)
- 4.3 Self-Join & GROUP BY
- 4.4 HAVING & Xếp hạng RANK()

## Phần 5: Truy vấn Nâng cao & Tập hợp

- 5.1 Subquery & Toán tử EXISTS
- 5.2 Bài toán "Tất cả" (NOT EXISTS)
- 5.3 COUNT(DISTINCT) & Tập chia rỗng
- 5.4 Phép toán tập hợp (Set Operators)

## Phần 6: Ràng buộc & Trigger T-SQL

- 6.1 Bảng ảo inserted & deleted
- 6.2 Trigger an toàn đa dòng
- 6.3 Phân tích sập logic biến vô hướng
- 6.4 Hiện thực Bảng tầm ảnh hưởng

## Lab 01: Xây dựng CSDL (DDL & RBTV)

Kiểu dữ liệu, NULL, PK/FK, CHECK/DEFAULT  
ALTER TABLE và circular-FK lifecycle

## Labs 02–04: DML, nâng cao và phân tích

- Lab 02 — DML, JOIN, NULL/date
- Lab 03 — Subquery, EXISTS, universal, set
- Lab 04 — GROUP, HAVING, RANK, TOP WITH TIES
- Fixture và expected-result contract

## Part 11–12 & Phụ lục

Debugging dictionary và trigger test  
Practical exam workflow (snapshot)  
Quick reference, provenance, expected results, checklist

## 0. Môi trường làm việc & Quy trình chạy Script SSMS

SQL Server Management Studio (SSMS) là công cụ giao diện đồ họa chính thức kết nối và gửi lệnh T-SQL tới SQL Server Database Engine. Hiểu rõ quy trình phân tích cú pháp, phân tách khối lệnh và quản lý giao dịch giúp loại bỏ hoàn toàn các lỗi vận hành thường gặp trong phòng thực hành và phòng thí.

### 0.1 Phân tách khối lệnh với GO

GO không phải là một câu lệnh T-SQL, mà là một từ khóa điều khiển lệnh (**command separator**) của SSMS để báo hiệu kết thúc một đợt lệnh (batch) và gửi toàn bộ đợt đó về máy chủ để thực thi.

#### Phạm vi của biến cục bộ

Mọi biến cục bộ khai báo bằng `DECLARE @x ...` chỉ tồn tại trong phạm vi của **duy nhất đợt lệnh đó**. Khi gặp lệnh `GO`, toàn bộ biến cục bộ trước đó sẽ bị hủy khỏi bộ nhớ.

```
DECLARE @MaKhoa VARCHAR(5) = 'CNTT';
SELECT * FROM GIAOVIEN WHERE MAKHOA = @MaKhoa;
GO -- Kết thúc Batch 1, @MaKhoa bị hủy

-- Lỗi Msg 137: Phải khai báo lại biến!
SELECT * FROM KHOA WHERE MAKHOA = @MaKhoa;
```

### 0.2 Checklist 60 giây trước khi bấm Execute (F5)

| Hạng mục                | Nguy cơ tiềm ẩn                                                           | Thao tác chuẩn mực                                                                |
|-------------------------|---------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| 1. Ngăn cảnh CSDL       | Vô tình tạo bảng/chèn dữ liệu vào CSDL hệ thống master.                   | Luôn đặt lệnh <code>USE TenDatabase;</code> GO ở đầu script.                      |
| 2. Định dạng ngày tháng | Lỗi chuyển đổi kiểu dữ liệu ngày khi hệ thống dùng định dạng Mỹ hoặc Anh. | Thiết lập <code>SET DATEFORMAT YMD;</code> để chuỗi 'YYYY-MM-DD' luôn hợp lệ.     |
| 3. Vùng chọn thực thi   | Bôi đen thiếu câu lệnh khiến SSMS chỉ chạy một đoạn cắt gây lỗi cú pháp.  | Bấm chuột ra ngoài hoặc nhấn <code>Ctrl+A</code> trước khi nhấn <code>F5</code> . |
| 4. Kiểm tra cú pháp     | Chạy trực tiếp câu lệnh lỗi có thể làm sai lệch dữ liệu thử nghiệm.       | Nhấn <code>Ctrl+F5</code> (Parse) để kiểm tra cú pháp trước khi chạy.             |

### 0.3 Kỹ thuật cô lập giao dịch kiểm thử (Safe Sandbox)

(Kỹ thuật an toàn vận hành trong thực nghiệm — Năm ngoài phạm vi kỹ năng cốt lõi IT004 theo chuẩn Phase A)

Khi kiểm thử câu lệnh sửa đổi dữ liệu (`INSERT`, `UPDATE`, `DELETE`), việc bao bọc mã trong khối giao dịch có `ROLLBACK` cho phép xem kết quả biến đổi mà không làm thay đổi dữ liệu gốc:

```
-- 1. Xem dữ liệu ban đầu trước khi thay đổi
SELECT MAGV, HOTEN, MUCLUONG FROM GIAOVIEN WHERE MAKHOA = 'CNTT';

BEGIN TRANSACTION;
-- 2. Thực hiện câu lệnh DML cần kiểm thử
UPDATE GIAOVIEN SET MUCLUONG = MUCLUONG * 1.1 WHERE MAKHOA = 'CNTT';

-- 3. Xem kết quả đã thay đổi ngay BÊN TRONG giao dịch
SELECT MAGV, HOTEN, MUCLUONG FROM GIAOVIEN WHERE MAKHOA = 'CNTT';

ROLLBACK TRANSACTION; -- Hoàn tác toàn bộ, hủy thay đổi

-- 4. Xác nhận dữ liệu gốc đã được khôi phục nguyên vẹn
SELECT MAGV, HOTEN, MUCLUONG FROM GIAOVIEN WHERE MAKHOA = 'CNTT';
```

## 1. Xây dựng CSDL & Ràng buộc Khai báo (DDL)

Ngôn ngữ định nghĩa dữ liệu (Data Definition Language - DDL) dùng để khởi tạo cấu trúc lưu trữ và các ràng buộc dữ liệu. Thiết kế DDL đúng ngay từ đầu giúp hệ thống tự động ngăn chặn dữ liệu rác ở tầng máy chủ.

### 1.1 Mini-Lab: Thiết kế bảng chuẩn mực

Xét hai quan hệ `KHACHHANG` và `SANPHAM` trong lược đồ Quản lý bán hàng chuẩn (căn cứ `TECH-A01`, `TECH-A03`):

```
CREATE DATABASE QuanLyBanHang;
GO
USE QuanLyBanHang;
GO

CREATE TABLE KHACHHANG (
    MAKH      CHAR(4)      NOT NULL,
    HOTEN     NVARCHAR(40) NOT NULL,
    DCHI      NVARCHAR(50),
    SODT      VARCHAR(20),
    NGSINH    SMALLDATETIME,
    NGDK      SMALLDATETIME,
    DOANHSON  MONEY        DEFAULT 0,
    CONSTRAINT PK_KHACHHANG PRIMARY KEY (MAKH),
    CONSTRAINT CK_KHACHHANG_NGAY CHECK (NGDK > NGSINH)
);

CREATE TABLE SANPHAM (
    MASP      CHAR(4)      NOT NULL,
    TENSP     NVARCHAR(40) NOT NULL,
    DVT       NVARCHAR(20),
    NUOCSX    NVARCHAR(40) DEFAULT N'Viet Nam',
    GIA       MONEY        CONSTRAINT CK_SANPHAM_GIA CHECK (GIA >= 500),
    CONSTRAINT PK_SANPHAM PRIMARY KEY (MASP)
);
```

#### Bẫy kiểu dữ liệu: NUMERIC(4,2)

Khi lưu trữ điểm số (thang 10), khai báo `NUMERIC(2,2)` chỉ cho phép giá trị tối đa là `0.99`. Để lưu được điểm `10.00`, bắt buộc phải khai báo `NUMERIC(4,2)` (tổng 4 chữ số, trong đó 2 chữ số phần thập phân).

#### Thứ tự thực thi DDL

**Tạo bảng:** Bảng cha (chứa PK) phải tạo trước bảng con (chứa FK).

**Xóa bảng:** Bảng con (chứa FK) phải xóa trước bảng cha; hoặc xóa FK constraint trước bằng `ALTER TABLE DROP CONSTRAINT`.

## 2. Thao tác Dữ liệu & Quy trình Kiểm chứng (DML)

Ngôn ngữ thao tác dữ liệu (Data Manipulation Language - DML) bao gồm INSERT, UPDATE, DELETE. Để đảm bảo tính chính xác, mọi thao tác DML cần được thực hiện qua quy trình kiểm chứng 3 bước.

### 2.1 Mini-Lab: Thao tác dữ liệu mẫu chuẩn

```
-- 1. INSERT đa dòng (Multi-row Value List)
INSERT INTO SANPHAM (MASP, TENSP, DVT, NUOCSX, GIA) VALUES
('BC01', N'But chi 2B', N'Cay', N'Viet Nam', 5000),
('BC02', N'But chi 4B', N'Cay', N'Nhat Ban', 12000),
('ST01', N'So tay A5', N'Quyen', N'Viet Nam', 35000);

-- 2. UPDATE có tính toán và điều kiện lọc
UPDATE SANPHAM
SET GIA = GIA * 1.05
WHERE NUOCSX = N'Viet Nam' AND DVT = N'Cay';

-- 3. DELETE có điều kiện
DELETE FROM SANPHAM
WHERE GIA < 6000 AND NUOCSX = N'Viet Nam';
```

### 2.2 Quy trình kiểm chứng DML an toàn (Verify Workflow)

(Kỹ thuật an toàn vận hành trong thực nghiệm — Năm ngoài phạm vi kỹ năng cốt lõi IT004 theo chuẩn Phase A)

Khi làm bài thực hành, tuyệt đối không chạy lệnh DML "mù". Hãy phân biệt rõ giữa Chế độ kiểm thử và Chế độ ghi nhận vĩnh viễn:

#### QUY TRÌNH KIỂM THỬ AN TOÀN (TEST SANDBOX)

- BƯỚC 1** Kiểm tra ban đầu: Chạy `SELECT * FROM Bang WHERE <DieuKien>` để đếm số dòng và xem giá trị trước khi sửa đổi.
- BƯỚC 2** Mở giao dịch & Chạy DML: `BEGIN TRAN;` rồi thực thi lệnh DML. Kiểm tra số dòng bị tác động (N row(s) affected).
- BƯỚC 3** Đối chiếu chi tiết BÊN TRONG giao dịch: Chạy lại lệnh `SELECT` để xác nhận giá trị mới đã được tính toán chính xác.
- BƯỚC 4** Hoàn tác an toàn: Chạy `ROLLBACK TRAN;` để hủy thay đổi; hoặc chạy `COMMIT TRAN;` nếu ở chế độ ghi vĩnh viễn (Persist Mode).

| MASP | TENSP      | DVT   | NUOCSX   | GIA (Trước) | GIA (Sau UPDATE)   |
|------|------------|-------|----------|-------------|--------------------|
| BC01 | But chi 2B | Cay   | Viet Nam | 5,000       | 5,250 (+5%)        |
| BC02 | But chi 4B | Cay   | Nhat Ban | 12,000      | 12,000 (Không đổi) |
| ST01 | So tay A5  | Quyen | Viet Nam | 35,000      | 35,000 (Khác DVT)  |

### 3. Kỹ thuật Truy vấn Nền tảng & Logic 3 Giá trị

Truy vấn lọc dữ liệu cơ bản trong SQL Server sử dụng mệnh đề `SELECT ... FROM ... WHERE`. Hiểu rõ logic 3 giá trị (True, False, Unknown) là điều kiện tiên quyết để tránh bẫy dữ liệu khi xử lý giá trị `NULL`.

#### 3.1 Cú pháp lọc chuỗi, đoạn & tập hợp

```
-- 1. LIKE: Tìm sản phẩm mã bắt đầu bằng 'B' và tận cùng bằng '01'
SELECT MASP, TENS, GIA FROM SANPHAM WHERE MASP LIKE 'B%01';

-- 2. BETWEEN: Lọc trong đoạn đóng [5000, 35000]
SELECT MASP, TENS, GIA FROM SANPHAM WHERE GIA BETWEEN 5000 AND 35000;

-- 3. IN: Thuộc tập giá trị rời rạc
SELECT MASP, TENS, NUOCSX FROM SANPHAM WHERE NUOCSX IN (N'Việt Nam', N'Nhật Bản');
```

#### 3.2 Logic 3 giá trị (3-Valued Logic) & Bẫy NULL

`NULL` thể hiện thông tin chưa biết hoặc không tồn tại, không phải là số 0 hay chuỗi rỗng ''. Mọi phép so sánh bằng toán tử thông thường với `NULL` (như `= NULL` hoặc `!= NULL`) đều trả về `UNKNOWN`, và mệnh đề `WHERE` sẽ loại bỏ tất cả các dòng có kết quả `UNKNOWN`.

| Toán hạng A | Toán hạng B | A AND B | A OR B  |
|-------------|-------------|---------|---------|
| TRUE        | UNKNOWN     | UNKNOWN | TRUE    |
| FALSE       | UNKNOWN     | FALSE   | UNKNOWN |
| UNKNOWN     | UNKNOWN     | UNKNOWN | UNKNOWN |

Bắt buộc dùng IS NULL

```
-- SAI: Có thể trả về 0 dòng nếu NULL/điều kiện không khớp
SELECT * FROM KHACHHANG WHERE SODT = NULL;

-- ĐÚNG: Cú pháp chuẩn SQL Server
SELECT * FROM KHACHHANG WHERE SODT IS NULL;
```

#### 3.3 Sắp xếp kết quả với ORDER BY

`ORDER BY` luôn nằm ở cuối của câu truy vấn. Khi sắp xếp theo thứ tự tăng dần ( `ASC` ), SQL Server coi `NULL` là giá trị nhỏ nhất và đặt ở đầu kết quả.

## 4. Phép kết (JOIN) & Dòng Xử lý Truy vấn Mức Logic

Phép kết kết nối các hàng từ hai hay nhiều bảng dựa trên cột chung. Hiểu rõ thứ tự xử lý logic giúp lập trình viên suy luận chính xác tập kết quả đầu ra trước khi công cụ tối ưu hóa (Query Optimizer) lập kế hoạch thực thi vật lý.

### 4.1 Worked Example: Truy vấn kết nối & Lọc khoảng ngày an toàn

**Yêu cầu:** In ra danh sách các hóa đơn mua hàng trong ngày 2024-10-15 gồm: Tên khách hàng, Số hóa đơn và Trị giá, sắp xếp theo Trị giá giảm dần.

```
SELECT KH.HOTEN, HD.SOHD, HD.TRIGIA
FROM HOADON HD
INNER JOIN KHACHHANG KH ON HD.MAKH = KH.MAKH
WHERE HD.NGHD >= '20241015' AND HD.NGHD < '20241016'
ORDER BY HD.TRIGIA DESC;
```

#### Kỹ thuật lọc ngày nửa mở (Half-Open Interval)

Viết `NGHD >= '20241015' AND NGHD < '20241016'` đảm bảo bao trọn toàn bộ các mốc thời gian từ 00:00:00 đến 23:59:59 khi cột có kiểu `SMALLDATETIME` / `DATETIME`, đồng thời duy trì khả năng tìm kiếm theo chỉ mục (SARGable).

### 4.2 Dòng xử lý truy vấn mức logic (Logical Processing Trace)

#### LOGICAL QUERY PROCESSING TRACE (THỨ TỰ LOGIC)

- FROM & JOIN** Ràng buộc bảng `HOADON` và kết nối với `KHACHHANG` trên điều kiện `HD.MAKH = KH.MAKH` để tạo tập dòng trung gian logic.
- WHERE** Áp dụng vị từ lọc ngày `HD.NGHD >= '20241015' AND HD.NGHD < '20241016'` để loại bỏ các dòng ngoài khoảng.
- SELECT** Chiếu u danh sách cột cần hiển thị: `KH.HOTEN`, `HD.SOHD`, `HD.TRIGIA`.
- ORDER BY** Sắp xếp tập kết quả cuối cùng theo cột `HD.TRIGIA` giảm dần. (Lưu ý: Optimizer có thể chọn kế hoạch thực thi vật lý khác như *Index Seek* / *Hash Match* để tối ưu hiệu năng).

### 4.3 Bảng kết quả mong đợi (Expected Output)

| HOTEN        | SOHD | TRIGIA    | Ghi chú đối chiếu                         |
|--------------|------|-----------|-------------------------------------------|
| Nguyen Van A | 1002 | 1,250,000 | Khớp MAKH = 'KH01', trong ngày 2024-10-15 |
| Le Van C     | 1005 | 890,000   | Khớp MAKH = 'KH07', trong ngày 2024-10-15 |
| Tran Thi B   | 1001 | 320,000   | Khớp MAKH = 'KH03', trong ngày 2024-10-15 |

4.4 Kỹ thuật Tự kết (Self-Join) & Gom nhóm (GROUP BY)

Tự kết (Self-Join) là phép kết một bảng với chính nó để giải quyết các mô hình quan hệ đệ quy (như nhân viên - người quản lý). Gom nhóm ( GROUP BY ) phân chia tập dữ liệu thành các nhóm độc lập để tính toán thống kê (căn cứ TECH-A02 ).

4.4.1 Self-Join: Quan hệ Nhân viên — Người quản lý

(Ví dụ tự biên soạn với lược đồ NHANVIEN(MANV, HOTEN, MAQL) để minh họa Self-Join rõ ràng):

```
-- Lấy mã nhân viên, tên nhân viên và tên người quản lý trực tiếp
SELECT NV.MANV, NV.HOTEN AS TenNhanVien, QL.HOTEN AS TenNguoiQuanLy
FROM NHANVIEN NV
LEFT JOIN NHANVIEN QL ON NV.MAQL = QL.MANV;
```

4.4.2 Cơ chế gom nhóm với GROUP BY

Mọi thuộc tính xuất hiện trong danh sách SELECT mà không nằm bên trong một hàm kết hợp (như COUNT , SUM , AVG , MIN , MAX ) đều **bắt buộc phải được khai báo trong mệnh đề GROUP BY** .

```
-- Thống kê số lượng sản phẩm và giá trung bình theo từng nước sản xuất
SELECT NUOCSX, COUNT(*) AS SoLuongSP, AVG(GIA) AS GiaTrungBinh
FROM SANPHAM
GROUP BY NUOCSX;
```

| NUOCSX    | SoLuongSP | GiaTrungBinh | Cơ chế xử lý                                            |
|-----------|-----------|--------------|---------------------------------------------------------|
| Viet Nam  | 12        | 24,500       | Gom toàn bộ 12 dòng có NUOCSX = 'Viet Nam', tính SUM/12 |
| Nhat Ban  | 6         | 85,000       | Gom toàn bộ 6 dòng có NUOCSX = 'Nhat Ban', tính SUM/6   |
| Singapore | 4         | 110,000      | Gom toàn bộ 4 dòng có NUOCSX = 'Singapore', tính SUM/4  |

**Sai lầm kinh điển: Lỗi Msg 8120**

Nếu viết SELECT NUOCSX, TENSX, COUNT(\*) FROM SANPHAM GROUP BY NUOCSX; , SQL Server sẽ báo lỗi Msg 8120 vì trong mỗi nhóm NUOCSX có nhiều TENSX khác nhau, máy chủ không thể biết lấy tên nào để hiển thị vào 1 ô kết quả duy nhất.

### 4.5 Lọc sau gom nhóm (HAVING) & Xếp hạng (RANK)

Sự khác biệt cốt lõi giữa `WHERE` và `HAVING` nằm ở thời điểm bộ lọc được áp dụng trong vòng đời truy vấn: `WHERE` lọc trên từng dòng dữ liệu thô, trong khi `HAVING` lọc trên các nhóm sau khi đã tính toán hàm kết hợp.

#### 4.5.1 Lọc nhóm với `HAVING`

**Yêu cầu:** Tìm các nước sản xuất có từ 3 sản phẩm trở lên với giá bán tối đa lớn hơn 50,000.

```
SELECT NUOCSX, COUNT(*) AS SoLuongSP, MAX(GIA) AS GiaCaoNhat
FROM SANPHAM
GROUP BY NUOCSX
HAVING COUNT(*) >= 3 AND MAX(GIA) > 50000;
```

#### 4.5.2 Xếp hạng với `RANK() OVER` & `TOP WITH TIES`

(Kỹ thuật phân tích xếp hạng khảo sát từ nguồn thực nghiệm `LOC-SQL-LAB04`, `LOC-HW-23520266-5`):

-- 1. Xếp hạng toàn cục bằng RANK()
SELECT MAKH, HOTEN, DOANHSO,
 RANK() OVER (ORDER BY DOANHSO DESC) AS
Hang
FROM KHACHHANG;

-- 2. Lấy Top 3 tính cả đồng hạng
SELECT TOP 3 WITH TIES MAKH, HOTEN, DOANHSO
FROM KHACHHANG
ORDER BY DOANHSO DESC;

| MAKH | HOTEN        | DOANHSO    | RANK() | TOP 3 WITH TIES | Giải thích cơ chế                                              |
|------|--------------|------------|--------|-----------------|----------------------------------------------------------------|
| KH01 | Nguyen Van A | 15,000,000 | 1      | Được chọn       | Hạng 1 duy nhất                                                |
| KH04 | Pham Van D   | 12,000,000 | 2      | Được chọn       | Hạng 2 đồng hạng (a)                                           |
| KH09 | Hoang Thi E  | 12,000,000 | 2      | Được chọn       | Hạng 2 đồng hạng (b)                                           |
| KH02 | Tran Thi B   | 9,500,000  | 4      | Bị loại         | Không bằng giá trị của dòng thứ 3 nằm trong TOP 3 (12,000,000) |
| KH07 | Le Van C     | 8,000,000  | 5      | Bị loại         | Vượt quá ngưỡng Top 3                                          |

**Nguyên tắc hoạt động của `WITH TIES`**

Mệnh đề `WITH TIES` chỉ bổ sung thêm các dòng có giá trị `ORDER BY` bằng với giá trị của dòng cuối cùng nằm trong ngưỡng `TOP N` (ở đây dòng thứ 3 có doanh số 12,000,000). Vì dòng thứ 4 có doanh số 9,500,000 nên không được chọn.

## 5. Truy vấn Lồng (Subquery) & Toán tử EXISTS

Truy vấn lồng là câu truy vấn `SELECT` được đặt bên trong một câu lệnh SQL khác. Phân biệt rõ truy vấn con độc lập và truy vấn con tương quan (Correlated Subquery) là chìa khóa để giải quyết các bài toán logic phức tạp (căn cứ `TECH-A07`).

### 5.1 Truy vấn con độc lập vs Truy vấn con tương quan

#### Độc lập (Self-Contained)

Chạy duy nhất 1 lần độc lập, trả về giá trị hoặc tập giá trị cho câu cha sử dụng (ví dụ: tìm sản phẩm có giá bán cao nhất).

```
SELECT MASP, TENSP, GIA
FROM SANPHAM
WHERE GIA = (
    SELECT MAX(GIA) FROM SANPHAM
);
```

#### Tương quan (Correlated)

Phụ thuộc mức logic vào giá trị từng dòng ngoài cùng; Optimizer có thể biến đổi thành Semi-Join/Apply thay vì lặp vật lý.

```
SELECT SP1.MASP, SP1.TENSP, SP1.GIA
FROM SANPHAM SP1
WHERE SP1.GIA = (
    SELECT MAX(SP2.GIA) FROM SANPHAM SP2
    WHERE SP2.NUOC SX = SP1.NUOC SX
);
```

### 5.2 Toán tử EXISTS / NOT EXISTS (Kiểm tra sự tồn tại)

Toán tử `EXISTS` trả về `TRUE` khi kết quả của truy vấn con không rỗng; danh sách cột trong mệnh đề `SELECT` của truy vấn con không quyết định giá trị trả về (chuẩn mực viết là `SELECT 1`). Về mặt logic, vị từ được thỏa mãn ngay khi tìm thấy dòng đầu tiên; kế hoạch thực thi vật lý cụ thể do Query Optimizer quyết định.

```
-- Tìm các sản phẩm chưa từng được bán trong bất kỳ hóa đơn nào
SELECT SP.MASP, SP.TENSP
FROM SANPHAM SP
WHERE NOT EXISTS (
    SELECT 1
    FROM CTHD CT
    WHERE CT.MASP = SP.MASP -- Tham chiếu tương quan với SP.MASP ở ngoài
);
```

#### CƠ CHẾ ĐÁNH GIÁ CỦA NOT EXISTS

- DÒNG 1** Xét sản phẩm `BC01`: Câu con quét bảng `CTHD` thấy có dòng (`SOHD: 1001, MASP: BC01`)  
\$\\rightarrow\$ `EXISTS` là `TRUE` \$\\rightarrow\$ `NOT EXISTS` là `FALSE` \$\\rightarrow\$ Loại `BC01`.
- DÒNG 2** Xét sản phẩm `BC05`: Ở mức logic, câu con xét các dòng `CTHD` và không có dòng nào chứa `BC05`  
\$\\rightarrow\$ `EXISTS` là `FALSE` \$\\rightarrow\$ `NOT EXISTS` là `TRUE` \$\\rightarrow\$ Giữ lại `BC05`.

## 5.3 Bài toán "Tất Cả" & Phép chia trong T-SQL

Điều kiện "tất cả" xuất hiện lặp lại trong corpus bài tập và đề thi đã khảo sát ở Phase A (nguồn LOC-SQL-LAB03, LOC-SQL-LAB04, PRAC-2024-2025-HK1-01); tài liệu TECH-A07 được dùng để chuẩn hóa cú pháp và ngữ nghĩa toán tử EXISTS.

### 5.3.1 Phương pháp chuẩn tắc: Double NOT EXISTS

Biến đổi logic: "Tìm khách hàng sao cho **KHÔNG TỒN TẠI** sản phẩm Singapore nào mà khách hàng đó **CHƯA MUA**."

```
SELECT KH.MAKH, KH.HOTEN
FROM KHACHHANG KH
WHERE NOT EXISTS (
    -- Tập chia: Quét toàn bộ sản phẩm do Singapore sản xuất
    SELECT * FROM SANPHAM SP
    WHERE SP.NUOCSX = 'Singapore'
    AND NOT EXISTS (
        -- Kiểm tra: Khách hàng KH đang xét có mua sản phẩm SP này không?
        SELECT * FROM CTHD CT
        INNER JOIN HOADON HD ON CT.SOHD = HD.SOHD
        WHERE HD.MAKH = KH.MAKH AND CT.MASP = SP.MASP
    )
);
```

#### TRACE SUY LUẬN LOGIC (LOGICAL CORRELATED TRACE)

##### 1. CÂU NGOÀI

Duyệt từng ứng viên khách hàng  $x \in \text{KHACHHANG}$ .

##### 2. CÂU GIỮA

Quét tập các sản phẩm yêu cầu  $y \in \text{SANPHAM}$  (có  $\text{NUOCSX} = \text{'Singapore'}$ ).

##### 3. CÂU TRONG

Kiểm tra xem cặp  $(x, y)$  có xuất hiện trong lịch sử mua hàng không. Nếu có ít nhất một  $yy$  mà  $xx$  chưa mua, câu giữa có kết quả  $\rightarrow$  câu ngoài bị loại.

### 5.3.2 Phương pháp so sánh số lượng: COUNT DISTINCT

```
SELECT HD.MAKH
FROM HOADON HD
INNER JOIN CTHD CT ON HD.SOHD = CT.SOHD
INNER JOIN SANPHAM SP ON CT.MASP = SP.MASP
WHERE SP.NUOCSX = 'Singapore'
GROUP BY HD.MAKH
HAVING COUNT(DISTINCT CT.MASP) = (
    SELECT COUNT(*) FROM SANPHAM WHERE NUOCSX = 'Singapore'
);
```

#### Xử lý trường hợp tập chia rỗng

Nếu CSDL không có sản phẩm Singapore nào: Double NOT EXISTS trả về **toàn bộ khách hàng** (đúng theo ngữ nghĩa toán học logic mệnh đề chân lý rỗng); trong khi phương pháp COUNT DISTINCT ở trên trả về 0 dòng do bảng kết nối bị rỗng trước khi gom nhóm.

## 5.4 Các Phép toán Tập hợp (Set Operators)

SQL Server hỗ trợ ba phép toán tập hợp chuẩn tấ c: `UNION` (Hợp), `EXCEPT` (Trừ) và `INTERSECT` (Giao). `UNION` loại dòng trùng, còn `UNION ALL` giữ dòng trùng; trong handbook, `INTERSECT` và `EXCEPT` dùng dạng mặc định, không có biế n thể `ALL` . Để thực hiện các phép toán này, hai tập truy vấ n phải thỏa mãn **điề u kiện khả hợp (Union-Compatibility)** (căn cứ `TECH-A08` ).

### 5.4.1 Điều kiện khả hợp trong T-SQL

- Hai câu lệnh `SELECT` phải có **cùng số lượng cột**.
- Các cột tương ứng ở cùng vị trí phải có **kiểu dữ liệu tương thích** (hoặc chuyển đổi ngắ m định được).
- Tên cột của tập kế t quả cuối cùng được lấ y theo **tên cột của câu `SELECT` đầ u tiên**.

### 5.4.2 Phân biệt UNION vs UNION ALL

```
-- UNION: SQL Server loại bỏ dòng trùng theo
mặc định
SELECT MAKH FROM HOADON WHERE YEAR(NGHD) = 2024
UNION
SELECT MAKH FROM KHACHHANG WHERE DOANHSo >
10000000;
```

```
-- UNION ALL: SQL Server giữ nguyên dòng trùng
SELECT MASP FROM CTHD WHERE SOHD = 1001
UNION ALL
SELECT MASP FROM CTHD WHERE SOHD = 1002;
```

### 5.4.3 Phép trừ (EXCEPT) & Phép giao (INTERSECT)

**Yêu cầu:** In ra mã các sản phẩm có hóa đơn mua trong năm 2023 nhưng chưa từng được mua trong năm 2024 .

```
SELECT CT.MASP
FROM CTHD CT INNER JOIN HOADON HD ON CT.SOHD = HD.SOHD
WHERE YEAR(HD.NGHD) = 2023

EXCEPT

SELECT CT.MASP
FROM CTHD CT INNER JOIN HOADON HD ON CT.SOHD = HD.SOHD
WHERE YEAR(HD.NGHD) = 2024;
```

| Tập truy vấn 1 (Năm 2023)  | Tập truy vấn 2 (Năm 2024) | Toán tử   | Kết quả trả về                     |
|----------------------------|---------------------------|-----------|------------------------------------|
| { BC01, BC02, ST01, ST02 } | { BC01, ST01, BB01 }      | EXCEPT    | { BC02, ST02 } (Chỉ có trong 2023) |
| { BC01, BC02, ST01, ST02 } | { BC01, ST01, BB01 }      | INTERSECT | { BC01, ST01 } (Có ở cả 2 năm)     |

## 6. Ràng buộc Khai báo & Cơ chế DML Trigger

Trong phạm vi các ràng buộc toàn vẹn liên bảng của IT004, **DML Trigger** là một cách hiện thực phù hợp khi các ràng buộc khai báo ( `CHECK` , `FOREIGN KEY` ) không biểu đạt được điều kiện (căn cứ `TECH-A04` , `TECH-A05` ).

### 6.1 Bảng so sánh phương án hiện thực RBTV

| Phương án                                                          | Phạm vi áp dụng                                                                                     | Đặc điểm kỹ thuật                                                                                  |
|--------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| <b>Khai báo (Declarative: <code>CHECK</code>, <code>FK</code>)</b> | Ràng buộc miêu tả giá trị trên 1 bảng; quan hệ khóa chính – khóa ngoại giữa 2 bảng.                 | Thực thi trực tiếp ở tầng máy chủ; không thể chứa subquery hoặc tham chiếu bảng ngoài.             |
| <b>Thủ tục (Procedural: <code>AFTER Trigger</code>)</b>            | Ràng buộc liên bảng, kiểm tra logic nghiệp vụ phức tạp, ràng buộc so sánh ngày tháng giữa các bảng. | Biểu đạt được logic phức tạp; yêu cầu thiết kế chuẩn dạng tập hợp để tránh quá tải và lỗi đa dòng. |

### 6.2 Hai bảng ảo chuyển tiếp: `inserted` & `deleted`

Khi sự kiện DML xảy ra, SQL Server tự động tạo hai bảng tạm trong bộ nhớ có cấu trúc cột y hệt bảng gốc (căn cứ `TECH-A05` ):

| Thao tác DML        | Bảng <code>inserted</code> chứa gì? | Bảng <code>deleted</code> chứa gì? |
|---------------------|-------------------------------------|------------------------------------|
| <code>INSERT</code> | Các dòng mới vừa được thêm vào      | Rỗng (Không có dòng nào)           |
| <code>DELETE</code> | Rỗng (Không có dòng nào)            | Các dòng cũ vừa bị xóa đi          |
| <code>UPDATE</code> | Các dòng mới sau khi sửa            | Các dòng cũ trước khi sửa          |

#### Bản chất của lệnh `UPDATE` trong SQL Server

SQL Server thực hiện `UPDATE` bằng cách xóa giá trị cũ (đưa vào `deleted` ) và chèn giá trị mới (đưa vào `inserted` ). Vì vậy, trong Trigger của `UPDATE` , cả hai bảng ảo đều chứa dữ liệu.

## 6.2 Thiết kế Trigger An toàn Đa dòng (Multi-Row Safety)

SQL Server là hệ quản trị xử lý theo tập hợp. Một câu lệnh `INSERT`, `UPDATE` hay `DELETE` có thể tác động cùng lúc lên hàng nghìn dòng. Trigger phải luôn được thiết kế để xử lý chính xác tình huống đa dòng này (căn cứ TECH-A06 ).

### 6.2.1 Phân tích sai lầm kinh điển: Biến vô hướng (Scalar Trigger)

#### CÁCH VIẾT SAI (Dễ sập logic)

```
-- SAI LẦM: Dùng biến vô hướng
DECLARE @NgàyHD SMALLDATETIME;
SELECT @NgàyHD = NGHD FROM inserted;
-- Nếu inserted có 100 dòng,
-- biến @NgàyHD chỉ nhận giá trị
-- của 1 dòng ngẫu nhiên duy nhất!
-- 99 dòng vi phạm khác bị bỏ Lọt!
```

#### CÁCH VIẾT ĐÚNG (Dạng tập hợp)

```
-- CHUẨN MỰC: Kết nối tập hợp
IF EXISTS (
    SELECT 1
    FROM inserted i
    INNER JOIN KHACHHANG kh
        ON i.MAKH = kh.MAKH
    WHERE i.NGHD < kh.NGDK
)
-- Bắt trọn vẹn mọi dòng vi phạm!
```

### 6.2.2 Mẫu Trigger chuẩn tắc dạng tập hợp

**Yêu cầu:** Ngày mua hàng ( `NGHD` ) của một hóa đơn phải lớn hơn hoặc bằng Ngày đăng ký ( `NGDK` ) của khách hàng đó.

```
CREATE TRIGGER trg_Check_HoaDon_NgayMua
ON HOADON
AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    -- Kiểm tra xem có BẤT KỲ dòng nào trong inserted vi phạm quy tắc hay không
    IF EXISTS (
        SELECT 1
        FROM inserted i
        INNER JOIN KHACHHANG kh ON i.MAKH = kh.MAKH
        WHERE i.NGHD < kh.NGDK
    )
    BEGIN
        RAISERROR (N'Lỗi: Ngày mua hàng (NGHD) không thể trước Ngày đăng ký thành viên (NGDK)!', 16,
1);
        ROLLBACK TRANSACTION; -- Hủy bỏ toàn bộ thao tác DML vi phạm
        RETURN;
    END
END;
GO
```

### 6.3 Hiện thực Bảng tầm ảnh hưởng trong T-SQL

Bảng tầm ảnh hưởng trong lý thuyết xác định chính xác các sự kiện DML trên từng quan hệ có nguy cơ gây vi phạm. Trong thực hành T-SQL, mỗi quan hệ có dấu `+` sẽ được cài đặt một Trigger tương ứng.

#### 6.3.1 Phân tích bối cảnh & Bảng tầm ảnh hưởng

**Ràng buộc:** "Trưởng khoa của một khoa phải là giáo viên thuộc chính khoa đó."

| Quan hệ  | Thêm (INSERT)               | Xóa (DELETE)                       | Sửa (UPDATE)  |
|----------|-----------------------------|------------------------------------|---------------|
| KHOA     | + (Cần kiểm tra TRUONGKHOA) | -                                  | +(TRUONGKHOA) |
| GIAOVIEN | -                           | + (Nếu xóa đúng GV là Trưởng khoa) | +(MAKHOA)     |

#### 6.3.2 Cài đặt Trigger & Đánh giá ngữ nghĩa tĩnh (Static Semantic Validation)

```
-- 1. Trigger trên KHOA: Kiểm tra khi chỉ định/thay đổi Trưởng khoa
CREATE TRIGGER trg_Khoa_CheckTruongKhoa ON KHOA AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    IF EXISTS (
        SELECT 1 FROM inserted k
        LEFT JOIN GIAOVIEN gv ON k.TRUONGKHOA = gv.MAGV AND k.MAKHOA = gv.MAKHOA
        WHERE k.TRUONGKHOA IS NOT NULL AND gv.MAGV IS NULL
    )
    BEGIN
        RAISERROR (N'Lỗi: Trưởng khoa được chỉ định phải là giáo viên thuộc chính khoa đó!', 16, 1);
        ROLLBACK TRAN;
    END;
END;
GO

-- 2. Trigger trên GIAOVIEN: Phân biệt chính xác xóa thực sự và sửa MAKHOA
CREATE TRIGGER trg_GiaoVien_CheckKhoa ON GIAOVIEN AFTER DELETE, UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    -- Nhánh DELETE thực sự: Có trong deleted nhưng KHÔNG có trong inserted
    IF EXISTS (
        SELECT 1 FROM deleted d
        INNER JOIN KHOA k ON d.MAGV = k.TRUONGKHOA
        LEFT JOIN inserted i ON d.MAGV = i.MAGV
        WHERE i.MAGV IS NULL
    )
    BEGIN
        RAISERROR (N'Lỗi: Không thể xóa giáo viên đang giữ chức vụ Trưởng khoa!', 16, 1);
        ROLLBACK TRAN; RETURN;
    END;

    -- Nhánh UPDATE MAKHOA: Kiểm tra khi MAKHOA của Trưởng khoa bị sửa đổi
    IF UPDATE(MAKHOA) AND EXISTS (
        SELECT 1 FROM inserted i
        INNER JOIN KHOA k ON i.MAGV = k.TRUONGKHOA
        WHERE i.MAKHOA IS NULL OR i.MAKHOA <> k.MAKHOA
    )
    BEGIN
        RAISERROR (N'Lỗi: Trưởng khoa phải thuộc chính khoa đó (không được để NULL hoặc chuyển khoa khác)!', 16, 1);
        ROLLBACK TRAN; RETURN;
    END;
END;
GO
```

## LAB 01 · DDL &amp; RBTV

## Khởi tạo CSDL và đọc một lược đồ trước khi viết mã

Lab 01 đi từ một thư mục trống để đến một schema có khóa, miền giá trị và quan hệ tham chiếu. Mục tiêu không phải gõ nhanh mà là biết vì sao từng đối tượng phải xuất hiện ở vị trí đó.

### Mục tiêu

- Tạo database `IT004_Training` mà không xóa dữ liệu ngoài phạm vi.
- Đọc PK, FK, kiểu dữ liệu và thứ tự phụ thuộc.
- Biết điểm dừng kiểm tra sau mỗi nhóm lệnh.

### Lược đồ liên quan

`tr_customers` → `tr_orders` → `tr_order_items` ← `tr_products`

Nguồn định hướng: tiến trình LOC-SQL-LAB01; kiểu cú pháp và ràng buộc đối chiếu TECH-A01, TECH-A03. Các tên `tr_*` và dữ liệu trong lab là bộ huấn luyện tự biên soạn.

#### Suy luận trước khi viết

Bảng cha phải tồn tại trước bảng con. Vì vậy `tr_customers` và `tr_products` đứng trước `tr_orders`, rồi mới đến `tr_order_items`.

#### Câu hỏi chuyên tiếp

Trước khi chạy, hãy đánh dấu trên giấy: cột nào là định danh, cột nào có thể rỗng, cột nào tham chiếu sang bảng khác?

## 1. Tạo database và chọn đúng ngữ cảnh

**Các bước:** mở SSMS, kết nối đúng instance, mở tệp practice/sql/00\_create\_training\_db.sql, chạy riêng batch tạo database rồi xác nhận tên database trong Object Explorer.

```
IF DB_ID(N'IT004_Training') IS NULL
    EXEC(N'CREATE DATABASE [IT004_Training]');
GO
USE IT004_Training;
GO
```

### Cách kiểm tra

```
SELECT DB_NAME() AS CurrentDatabase;
SELECT name, state_desc
FROM sys.databases
WHERE name = N'IT004_Training';
```

| Kiểm tra  | Kết quả mong đợi từ script                       |
|-----------|--------------------------------------------------|
| DB_ID     | Có một dòng với trạng thái <code>ONLINE</code> . |
| DB_NAME() | IT004_Training; nếu là master thì chưa chạy USE. |

#### Lỗi thường gặp: chạy nhầm database

Msg 208 sau đó thường không phải do bảng “mất”, mà do cửa sổ query đang ở master. Đọc dòng DB\_NAME() trước khi sửa câu lệnh.

Nguồn / provenance note: LOC-SQL-LAB01 cho trình tự tạo database; TECH-A01 cho CREATE TABLE; quy trình kiểm tra là thực hành biên soạn riêng, không phải quy tắc chấm điểm của UIT.

## 2. Kiểu dữ liệu, NULL và tên ràng buộc

| Cột           | Kiểu trong fixture | Lý do thực hành                                              |
|---------------|--------------------|--------------------------------------------------------------|
| CustomerId    | CHAR(4)            | Mã ổn định, độ dài cố định; là PK.                           |
| FullName      | NVARCHAR(60)       | Giữ Unicode cho tên tiếng Việt; dùng tiền tố N ở literal.    |
| RegisteredOn  | DATE               | Chỉ cần ngày, không giả định giờ.                            |
| OrderDate     | DATETIME2(0)       | Giữ cả giờ để minh họa khoảng nửa kín.                       |
| Price, Salary | DECIMAL(10,2)      | Tiền tệ có phần lẻ; tránh phụ thuộc cách định dạng hiển thị. |

**NULL là trạng thái chưa biết, không phải 0**

HeadEmployeeId được phép NULL ở thời điểm tạo phòng ban. Dùng IS NULL để kiểm tra; không dùng = NULL.

**Quy ước đặt tên**

PK\_tr\_products, FK\_tr\_items\_product, CK\_tr\_items\_qty cho biết loại và đối tượng. Tên rõ giúp đọc Msg 547 hoặc Msg 2627 mà không phải đoán.

**Bài tự làm**

Thêm cột Phone kiểu VARCHAR(15) vào bản nháp, rồi giải thích vì sao không nên dùng INT cho số điện thoại.

Nguồn định hướng: TECH-A01, TECH-A03; việc chọn kích thước là quyết định minh họa cho fixture, không phải chuẩn duy nhất.

### 3. PRIMARY KEY và khóa phức hợp

**Suy luận:** PK phải định danh duy nhất mỗi dòng và không nhận NULL. Với chi tiết đơn hàng, một sản phẩm chỉ xuất hiện một lần trong cùng đơn, nhưng có thể xuất hiện ở đơn khác.

```
CREATE TABLE dbo.tr_order_items (  
    OrderId INT NOT NULL,  
    ProductId CHAR(4) NOT NULL,  
    Qty INT NOT NULL,  
    UnitPrice DECIMAL(10,2) NOT NULL,  
    CONSTRAINT PK_tr_order_items PRIMARY KEY (OrderId, ProductId)  
);
```

| Khóa                                          | Ý nghĩa trong fixture                             | Thử nghiệm sai                       |
|-----------------------------------------------|---------------------------------------------------|--------------------------------------|
| PK_tr_customers(CustomerId)                   | Mỗi khách một mã.                                 | Chèn lại C001 → Msg 2627.            |
| PK_tr_order_items(OrderId, ProductId)         | Một cặp đơn–sản phẩm là duy nhất.                 | Chèn lại (1001, P001) → vi phạm PK.  |
| PK_tr_results(StudentId, CourseId, AttemptNo) | Cho phép nhiều lần thi, nhưng số lần không trùng. | Chèn lại (S001,C101,2) → vi phạm PK. |

TRACE SUY LUẬN LOGIC

BƯỚC 1

Xác định “một dòng” là gì: một dòng chi tiết = một sản phẩm trong một đơn.

BƯỚC 2

Chọn các thuộc tính đủ để định danh: (OrderId, ProductId).

BƯỚC 3

Đặt PRIMARY KEY, sau đó mới tạo FK sang cha.

## 4. FOREIGN KEY và thứ tự phụ thuộc

FK bảo đảm giá trị con phải tồn tại ở khóa cha. Đây là liên kết dữ liệu, không chỉ là “dòng trang trí” trong schema.

```
CONSTRAINT FK_tr_orders_customer
  FOREIGN KEY (CustomerId)
  REFERENCES dbo.tr_customers(CustomerId)

CONSTRAINT FK_tr_items_order
  FOREIGN KEY (OrderId)
  REFERENCES dbo.tr_orders(OrderId)
```

### THỨ TỰ CHẠY SCRIPT

- 1. CHA** tr\_customers, tr\_products không phụ thuộc bảng huấn luyện khác.
- 2. CON** tr\_orders cần khách; tr\_order\_items cần đơn và sản phẩm.
- 3. KIỂM** Chỉ nạp child row sau khi parent row tồn tại.

### Cách kiểm tra FK

```
SELECT fk.name, OBJECT_NAME(fk.parent_object_id) AS ChildTable,
       OBJECT_NAME(fk.referenced_object_id) AS ParentTable
FROM sys.foreign_keys AS fk
WHERE fk.name LIKE 'FK_tr_%';
```

#### Lỗi thường gặp: chèn con trước cha

Thử chèn OrderId 1999 với CustomerId C999 sẽ nhận Msg 547. Cách sửa là bổ sung parent hợp lệ hoặc sửa mã con; không tắt FK để “cho chạy”.

Nguồn định hướng: TECH-A01, LOC-SQL-LAB01; lỗi 547 được dùng như bài kiểm tra trong fixture, không phải kết quả chạy đã xác nhận ở môi trường này.

## 5. CHECK, DEFAULT và miền giá trị

CHECK mô tả miền giá trị ngay tại database. Nó không thay thế việc kiểm tra nghiệp vụ liên bảng; với điều kiện liên bảng, cần FK hoặc trigger.

| Ràng buộc          | Ví dụ                          | Ý nghĩa                                    |
|--------------------|--------------------------------|--------------------------------------------|
| Giá dương          | CHECK (Price > 0)              | Không nhận giá 0 hoặc âm.                  |
| Trạng thái hữu hạn | CHECK (Status IN (...))        | Chặn chính tả và giá trị ngoài tập.        |
| Giá trị mặc định   | DEFAULT (1)                    | Chỉ dùng khi người gọi không cung cấp cột. |
| Điểm               | CHECK (Score BETWEEN 0 AND 10) | 10.00 hợp lệ; NUMERIC(2,2) không đủ miền.  |

### Kiểm tra âm thầm trước khi INSERT

```
SELECT CASE
    WHEN @Score BETWEEN 0 AND 10 THEN 'accepted'
    ELSE 'reject before write'
END;
```

Suy luận

Kiểm tra trước giúp thông báo thân thiện; CHECK vẫn là lớp bảo vệ cuối nếu một ứng dụng khác ghi vào database.

### Bài tự làm

Viết một CHECK cho Segment chỉ nhận A/B/C, sau đó nêu một giá trị cố ý vi phạm và thông báo dự kiến.

Nguồn / provenance note: TECH-A03 cho CHECK; ví dụ và tên đối tượng thuộc bộ huấn luyện tự biên soạn.

## 6. ALTER TABLE: thay đổi có kiểm soát

Trong phòng lab, sửa schema là một thao tác có rủi ro. Đọc dữ liệu hiện có, kiểm tra phụ thuộc, rồi mới thêm cột hoặc constraint.

```
ALTER TABLE dbo.tr_products
    ADD SKU VARCHAR(12) NULL;

ALTER TABLE dbo.tr_products
    ADD CONSTRAINT UQ_tr_products_sku UNIQUE (SKU);

ALTER TABLE dbo.tr_products
    DROP CONSTRAINT UQ_tr_products_sku;
```

| Trước khi sửa     | Câu hỏi cần trả lời                                         |
|-------------------|-------------------------------------------------------------|
| Thêm cột NOT NULL | Dòng cũ lấy giá trị nào? Có cần DEFAULT tạm thời không?     |
| Thêm FK           | Có dòng mô`côi đang tồn tại không?                          |
| Đổi kiểu          | Giá trị cũ có chuyển được không? Có mất độ chính xác không? |

### Không đổi kiểu để che lỗi dữ liệu

Nếu cột chứa chuỗi không chuyển được sang số, hãy lập danh sách vi phạm bằng TRY\_CONVERT, xử lý từng nhóm rồi mới ALTER COLUMN.

### Cách kiểm tra

```
SELECT c.name, t.name AS DataType, c.max_length, c.is_nullable
FROM sys.columns c JOIN sys.types t ON c.user_type_id = t.user_type_id
WHERE c.object_id = OBJECT_ID(N'dbo.tr_products');
```

Nguồn định hướng: LOC-SQL-LAB01; câu lệnh ALTER trong trang là ví dụ chuyển giao, không bắt buộc chạy trong seed chính.

## 7. Xử lý phụ thuộc vòng KHOA — NHÂN VIÊN

Hai bảng tr\_departments và tr\_employees có vòng: nhân viên cần phòng ban, phòng ban có thể trở trưởng phòng. Tách “tạo cấu trúc” khỏi “gán dữ liệu” để vòng không chặn việc khởi tạo.

### CÁC BƯỚC PHÁ VÒNG

- A** Tạo tr\_departments với HeadEmployeeId NULL, chưa thêm FK head.
- B** Tạo tr\_employees; FK DeptId trở về departments.
- C** Thêm FK HeadEmployeeId sau khi bảng employee đã tồn tại.
- D** Nạp nhân viên, rồi UPDATE trưởng phòng trong seed.

```
ALTER TABLE dbo.tr_departments
  ADD CONSTRAINT FK_tr_departments_head
  FOREIGN KEY (HeadEmployeeId)
  REFERENCES dbo.tr_employees(EmployeeId);
```

### Cách kiểm tra

Sau seed, truy vấn JOIN departments với employees và xác nhận E001 thuộc D001, E004 thuộc D002. Nếu u không khớp, chưa được xem là hoàn tất.

Nguồn định hướng: LOC-SQL-LAB01 và thực hành circular-FK tự biên soạn trong 01\_schema.sql/02\_seed.sql.

## 8. Một vòng chạy DDL có điểm dừng

1. Chạy 00\_create\_training\_db.sql.
2. Chạy 01\_schema.sql từng batch.
3. Liệt kê bảng và constraint.
4. Chỉ chuyển sang Lab 02 khi các tên bảng đúng.

```
SELECT name FROM sys.tables WHERE name LIKE 'tr_%' ORDER BY name;

SELECT name, type_desc
FROM sys.objects
WHERE parent_object_id > 0
AND name LIKE '%tr_%';
```

| Checkpoint | PASS khi                                             | Nếu FAIL                               |
|------------|------------------------------------------------------|----------------------------------------|
| Bảng       | 10 bảng tr_* xuất hiện.                              | Kiểm tra batch trước đó và DB context. |
| PK         | Mỗi bảng có khóa phù hợp.                            | Đọc lại định nghĩa ở 01_schema.sql.    |
| FK         | Các quan hệ cha-con hiển thị trong sys.foreign_keys. | Kiểm tra thứ tự và dữ liệu mô`côi.     |

### Bài tự làm chuyển Lab 02

Viết một truy vấn kiểm tra mọi FK của tr\_order\_items và giải thích vì sao bảng này phải được nạp sau orders/products.

Validation mode: STATIC — script được kiểm tra tên đối tượng bằng bộ kiểm tra đi kèm; chưa tuyên bố đã chạy SQL Server trong môi trường này.

## Tôi có thể...

- tạo IT004\_Training và chứng minh cửa sổ query đang ở đúng database;
- giải thích vì sao DATETIME2 và DATE được dùng ở hai cột khác nhau;
- chọn PK đơn hoặc PK phức hợp từ câu mô tả “một dòng là gì”;
- vẽ thứ tự parent → child trước khi chạy DDL;
- nêu nguyên nhân của lỗi FK, CHECK, duplicate PK;
- xử lý circular FK bằng cách trì hoãn constraint/gán head;
- đọc danh sách constraint từ catalog view thay vì đoán.

## Bài tự làm tổng hợp

Tạo bảng tr\_lab\_notes(StudentId, NoteNo, Body) với PK phức hợp, Body NOT NULL và FK về tr\_students. Viết thêm truy vấn kiểm tra orphan (nếu có) trước khi tạo FK.

### Kết quả mong đợi

Bài làm đạt khi bạn có thể trình bày được: mục tiêu → lược đồ → suy luận → mã → kiểm tra → lỗi → sửa. Không cần tạo thêm dữ liệu ngoài fixture.

Nguồn: tổng hợp LOC-SQL-LAB01, TECH-A01, TECH-A03; không sao chép mã nguồn lab gốc.

## LAB 02 · DML &amp; SELECT

## Nạp dữ liệu, kiểm tra lỗi và đọc kết quả

Lab 02 dùng đúng fixture `IT004_Training`. Mỗi truy vấn đều có mã Bxx để đối chiếu với `practice/sql/03_queries_basic.sql`; dữ liệu trong `02_seed.sql` là dữ liệu nhỏ, xác định và có thể nạp lại.

### Mục tiêu

- Chèn theo thứ tự bảng cha → bảng con.
- Phân biệt INSERT, UPDATE, SELECT và điều kiện lọc.
- Đọc lỗi khóa, CHECK, ngày và kiểu dữ liệu trước khi sửa.

`tr_customers` → `tr_orders` → `tr_order_items` ← `tr_products`

Provenance: tiến trình thao tác đối chiếu LOC-SQL-LAB02; trình tự DML đối chiếu LOC-SQL-LAB02. Fixture là dữ liệu huấn luyện tự biên soạn.

### Checkpoint

Tôi có thể nêu bảng cha của từng FK và dự đoán lệnh INSERT nào phải chạy trước.

## 1. Tải dữ liệu theo dependency

Chạy `01_schema.sql` rồi `02_seed.sql`. Nếu chạy lại, schema được tạo lại có chủ đích; không trộn một nửa seed với dữ liệu cũ.

```
INSERT INTO dbo.tr_customers(CustomerId, FullName, City, RegisteredOn, CreditLimit, Segment)
VALUES ('C001', N'An Phạm', N'Thủ Đức', '20240115', 5000.00, 'A');

INSERT INTO dbo.tr_orders(OrderId, CustomerId, OrderDate, Status)
VALUES (1001, 'C001', '2024-10-15T09:00:00', 'PAID');

INSERT INTO dbo.tr_order_items(OrderId, ProductId, Qty, UnitPrice)
VALUES (1001, 'P001', 2, 120000.00);
```

| Nhóm                                   | Thứ tự | Kiểm tra                          |
|----------------------------------------|--------|-----------------------------------|
| Customers, products                    | 1      | PK tồn tại, số dòng đúng fixture. |
| Orders, departments, students, courses | 2      | FK cha đã có.                     |
| Items, employees, results              | 3      | FK và khóa ghép hợp lệ.           |

Kết quả mong đợi chỉ áp dụng khi chạy nguyên bộ seed trên schema mới; nếu dữ liệu đã bị sửa, hãy dùng `reset.sql`.

## 2. Thử nghiệm lỗi có chủ đích

Chạy từng khối riêng trong transaction rồi rollback. Không biếm lỗi minh họa thành dữ liệu của fixture.

```
BEGIN TRAN;  
INSERT INTO dbo.tr_order_items(OrderId, ProductId, Qty, UnitPrice)  
VALUES (9999, 'P001', 1, 10.00); -- FK order: Msg 547  
ROLLBACK;  
  
BEGIN TRAN;  
INSERT INTO dbo.tr_products(ProductId, ProductName, Category, Country, Price, Stock, IsActive)  
VALUES ('P001', N'Trùng mã', 'DB', N'Việt Nam', 1.00, 1, 1); -- PK: Msg 2627  
ROLLBACK;
```

```
BEGIN TRAN;  
INSERT INTO dbo.tr_products(ProductId, ProductName, Category, Country, Price, Stock, IsActive)  
VALUES ('PX01', N'Giá âm', 'DB', N'Việt Nam', -5.00, 1, 1); -- CHECK: Msg 547  
ROLLBACK;
```

### Đọc thông báo, không đoán

Msg 547 có thể là FK hoặc CHECK. Đọc tên constraint trong thông báo và đối chiếu `01_schema.sql`.

### 3. Sửa dữ liệu với điều kiện đủ hẹp

Trước UPDATE, chạy SELECT cùng WHERE để biết số dòng bị tác động. Cột `OrderDate` là `DATETIME2` ; dùng khoảng nửa kín để không bỏ sót giờ cuối ngày.

```
SELECT OrderId, Status
FROM dbo.tr_orders
WHERE OrderDate >= '2024-10-01'
      AND OrderDate < '2024-11-01';

UPDATE dbo.tr_orders
SET Status = 'CANCELLED'
WHERE OrderId = 1002;

SELECT OrderId, Status
FROM dbo.tr_orders
WHERE OrderId = 1002;
```

| Biểu thức                                             | Ý nghĩa                                                 |
|-------------------------------------------------------|---------------------------------------------------------|
| <code>City IS NULL</code>                             | Tìm giá trị chưa biết; không dùng <code>= NULL</code> . |
| <code>OrderDate &gt;= d AND OrderDate &lt; d+1</code> | Khoảng thời gian [d, d+1), an toàn với phần giờ.        |
| <code>Qty &gt; 0</code>                               | Điều kiện CHECK trong fixture.                          |

#### Thử mình

Viết UPDATE chỉ đổi trạng thái đơn 1003. SELECT trước và sau phải trả cùng một khóa đơn.

### 4. SELECT, WHERE, LIKE, BETWEEN

Projection chọn cột; selection chọn dòng. B01 là truy vấn canonical trên tr\_customers.

```
SELECT CustomerId, FullName, City
FROM dbo.tr_customers
WHERE Segment IN ('A', 'B')
ORDER BY CustomerId;
```

```
SELECT CustomerId, FullName
FROM dbo.tr_customers
WHERE FullName LIKE N'%An%';

SELECT OrderId, OrderDate
FROM dbo.tr_orders
WHERE OrderDate >= '2024-10-15'
      AND OrderDate < '2024-10-18';
```

| Mã  | Kiểm tra tính                         | Giá trị then chốt                |
|-----|---------------------------------------|----------------------------------|
| B01 | Segment A/B trên tr_customers.        | C001, C002, C003 (3 dòng).       |
| B02 | Khoảng ngày [2024-10-15, 2024-10-18). | 1001, 1002, 1003, 1004 (4 dòng). |

Expected rows được xác định từ 02\_seed.sql ; nếu đổi seed, cập nhật mã ví dụ thay vì ghi kết quả cũ.

## 5. Nối bảng và giữ dòng không khớp

```
SELECT o.OrderId, c.FullName, p.ProductName, i.Qty
FROM dbo.tr_orders AS o
JOIN dbo.tr_customers AS c ON c.CustomerId = o.CustomerId
JOIN dbo.tr_order_items AS i ON i.OrderId = o.OrderId
JOIN dbo.tr_products AS p ON p.ProductId = i.ProductId
ORDER BY o.OrderId, i.ProductId;
```

```
SELECT c.CustomerId, c.FullName, o.OrderId
FROM dbo.tr_customers AS c
LEFT JOIN dbo.tr_orders AS o ON o.CustomerId = c.CustomerId
ORDER BY c.CustomerId, o.OrderId;
```

| Join           | Dòng giữ lại                        | Điểm kiểm tra                      |
|----------------|-------------------------------------|------------------------------------|
| INNER JOIN B03 | Chỉ cặp có đơn, chi tiết, sản phẩm. | Không xuất hiện sản phẩm chưa bán. |
| LEFT JOIN B04  | Tất cả khách hàng.                  | C005 vẫn có dòng với OrderId NULL. |

### NULL ở phía phải

NULL của cột o.OrderId ở B04 là dấu hiệu “không có dòng khớp”, không phải đơn hàng có mã rỗng.

## 6. Self join và UNION tương thích

```
SELECT e.EmployeeId, e.FullName,
       m.FullName AS ManagerName
FROM   dbo.tr_employees AS e
LEFT JOIN dbo.tr_employees AS m
      ON m.EmployeeId = e.ManagerId
ORDER BY e.EmployeeId;
```

```
SELECT CustomerId AS EntityId
FROM   dbo.tr_customers
UNION
SELECT StudentId
FROM   dbo.tr_students;
```

Hai nhánh UNION có cùng số cột và kiểu tương thích. Đây là minh họa tổng quát trên hai quan hệ độc lập; không gán chúng vào cùng một nghiệp vụ.

| Toán tử            | Đặc tính                                                  |
|--------------------|-----------------------------------------------------------|
| UNION              | SQL Server gộp và loại dòng trùng.                        |
| UNION ALL          | SQL Server giữ dòng trùng.                                |
| INTERSECT / EXCEPT | Trong handbook dùng dạng mặc định; không có biến thể ALL. |
| Self join          | Một bảng đóng hai vai; mọi cột nên có alias.              |

B05 nằm trong `03_queries_basic.sql` ; self join dùng fixture nhân viên. UNION đối chiếu TECH-A08.

## 7. Quy trình phục hồi và tự kiểm

1. Đóng các cửa sổ query đang giữ transaction.
2. Chạy `reset.sql` từ thư mục `practice/sql` bằng SQLCMD mode hoặc mở từng tệp theo thứ tự.
3. Kiểm tra số dòng và các khóa mẫu.

```
SELECT 'customers' AS TableName, COUNT_BIG(*) AS RowCount FROM dbo.tr_customers
UNION ALL SELECT 'orders', COUNT_BIG(*) FROM dbo.tr_orders
UNION ALL SELECT 'order_items', COUNT_BIG(*) FROM dbo.tr_order_items;
```

| Bảng           | Số dòng fixture | Khóa mẫu      |
|----------------|-----------------|---------------|
| tr_customers   | 5               | C001...C005   |
| tr_products    | 7               | P007 chưa bán |
| tr_orders      | 5               | 1001...1005   |
| tr_order_items | 11              | 1001/P001     |

### Tôi có thể...

giải thích vì sao seed phải nạp parent trước child, dùng ngày nửa kín, và chọn LEFT JOIN khi cần giữ khách chưa có đơn.

## 8. Bài tự làm có kiểm chứng

### Nhiệm vụ A

Tìm khách hàng chưa có đơn, chỉ dùng LEFT JOIN và điều kiện NULL ở phía phải. Ghi rõ cột nào là khóa của bảng phải.

### Nhiệm vụ B

Chèn một dòng sản phẩm hợp lệ, sau đó chèn một order\_item tham chiếu dòng đó. Chụp lại hai câu SELECT xác nhận.

### Nhiệm vụ C

Viết truy vấn UNION giữa mã khách và mã sinh viên. Nêu số cột, kiểu dữ liệu và ý nghĩa của việc loại bản sao.

### Checklist trước khi chuyển Lab 03

- Đã chạy đúng database.
- Đã phân biệt NULL với o/chuỗi rỗng.
- Đã kiểm tra số dòng bị UPDATE.
- Đã giữ nguyên seed để expected rows còn đồ i chiếu u được.

Mã bài B01–B05 được ánh xạ sang script cơ bản; không phải đề thi chính thức. Xem phần source map ở phụ lục.

## LAB 03 · SUBQUERY &amp; SET

## Từ câu hỏi tiếng Việt đến truy vấn nhiều tầng

Lab 03 giữ nguyên fixture `tr_*` và tập trung vào cách tách miền ứng viên, điều kiện tồn tại, và phép chia. Mỗi bài có một chuỗi suy luận trước khi có SQL.

### Mục tiêu

- Viết scalar, correlated và existence subquery.
- Phân biệt “có ít nhất một”, “không có” và “tất cả”.
- Giải thích UNION/INTERSECT/EXCEPT và CASE mà không dựa vào kế hoạch vật lý.

### Nguyên tắc trace

TRACE SUY LUẬN LOGIC mô tả quan hệ và kết quả logic; trình tối ưu có thể chọn kế hoạch thực thi khác.

TECH-A07 cho EXISTS và TECH-A08 cho set operators; các câu A01–A07 là bài huấn luyện độc lập, ánh xạ vào `04_queries_advanced.sql`.

### Tôi có thể...

gạch dưới cụm “tất cả”, “chưa từng”, “lớn nhất” trong đề trước khi chọn dạng subquery.

## 1. Giá trị lớn nhất và scalar subquery

```
SELECT ProductId, ProductName, Price
FROM dbo.tr_products
WHERE Price = (SELECT MAX(Price) FROM dbo.tr_products);
```

### TRACE SUY LUẬN LOGIC · A01

- 1 Miền ứng viên: mọi dòng tr\_products.
- 2 Subquery trả một giá trị MAX(Price).
- 3 Giữ dòng có giá trị đó; fixture có P001 là duy nhất.

| Kiểm tra     | Kỳ vọng                          |
|--------------|----------------------------------|
| Cardinality  | Scalar subquery trả một giá trị. |
| Expected key | P001 (1 dòng, Price 1200).       |

### Msg 512

Nếu subquery dùng như scalar trả nhiều dòng, SQL Server báo Msg 512. Đổi sang IN, EXISTS hoặc gom nhóm đúng ý nghĩa.

Expected value lấy từ seed; không ghi tên sản phẩm nếu chưa chạy runtime.

## 2. Giá lớn nhất theo quốc gia

```
SELECT p.ProductId, p.ProductName, p.Country, p.Price
FROM dbo.tr_products AS p
WHERE p.Price = (SELECT MAX(p2.Price) FROM dbo.tr_products AS p2 WHERE p2.Country = p.Country);
```

Subquery tham chiếu `p.Country`, nên giá trị cực đại được tính theo từng quốc gia.

| Bước     | Câu hỏi kiểm soát                                      |
|----------|--------------------------------------------------------|
| Miền     | Dòng ngoài p nào đang xét?                             |
| Liên kết | <code>p2.Country = p.Country</code> có đúng khóa nhóm? |
| Kết quả  | P001, P003, P005 trong fixture (3 dòng).               |

### Thử mình

Đổi MAX thành MIN và diễn giải lại câu hỏi.

Correlated subquery mô tả semantics logic; không suy ra số lần quét hay vòng lặp vật lý.

### 3. “Có ít nhất một” bằng EXISTS

```
SELECT c.CustomerId, c.FullName
FROM dbo.tr_customers AS c
WHERE EXISTS (SELECT 1 FROM dbo.tr_orders AS o WHERE o.CustomerId = c.CustomerId);
```

EXISTS đúng khi subquery có ít nhất một dòng; SELECT list bên trong không được trả ra ngoài. `SELECT 1` làm rõ mục đích, không phải cam kết về short-circuit vật lý.

```
SELECT p.ProductId, p.ProductName
FROM dbo.tr_products AS p
WHERE NOT EXISTS (SELECT 1 FROM dbo.tr_order_items AS i WHERE i.ProductId = p.ProductId);
```

**A03****A04**

Khách có đơn

Sản phẩm chưa bán; seed có P007

#### Kiểm tra

Thay EXISTS bằng JOIN rồi giải thích nguyên cơ lập dòng nếu một khách có nhiều đơn.

## 4. “Đã đạt tất cả” bằng Double NOT EXISTS

```
SELECT s.StudentId, s.FullName
FROM dbo.tr_students AS s
WHERE NOT EXISTS (SELECT 1 FROM dbo.tr_courses AS c WHERE NOT EXISTS (SELECT 1 FROM dbo.tr_results AS
r WHERE r.StudentId = s.StudentId AND r.CourseId = c.CourseId));
```

### TRACE SUY LUẬN LOGIC · “TẤT CẢ”

- 1 Miệ`n ứng viên: sinh viên.
- 2 Tìm khóa học còn thiế`u đố`i với từng sinh viên.
- 3 Giữ sinh viên không có khóa học còn thiế`u.

Empty divisor: nếu tr\_courses rỗng, điều kiện đúng theo logic vacuous truth. Nếu nghiệp vụ không muốn vậy, thêm điều kiện COUNT.

## 5. Kiểm tra “tất cả” bằng đếm tập

```
SELECT r.StudentId
FROM dbo.tr_results AS r
GROUP BY r.StudentId
HAVING COUNT(DISTINCT r.CourseId) = (SELECT COUNT(*) FROM dbo.tr_courses);
```

COUNT DISTINCT tránh retake làm tăng số lượng giả. Cách đếm này cần xem xét miền ứng viên: sinh viên chưa có dòng kết quả sẽ không xuất hiện.

| Biến thể         | Rủi ro                        | Sửa                             |
|------------------|-------------------------------|---------------------------------|
| COUNT(*)         | Retake đếm thừa.              | COUNT(DISTINCT CourseId).       |
| GROUP BY results | Bỏ sinh viên chưa có kết quả. | LEFT JOIN từ tr_students.       |
| Divisor rỗng     | 0 = 0 có thể giữ mọi nhóm.    | Nêu semantics hoặc thêm EXISTS. |

### Thử mình

Viết lại A05 bằng Double NOT EXISTS và so sánh hai miền ứng viên.

## 6. UNION, INTERSECT, EXCEPT

```
SELECT CustomerId AS Id FROM dbo.tr_customers
INTERSECT
SELECT StudentId FROM dbo.tr_students;

SELECT ProductId FROM dbo.tr_products
EXCEPT
SELECT ProductId FROM dbo.tr_order_items;
```

INTERSECT giữ giá trị ở cả hai tập; EXCEPT giữ giá trị chỉ ở vế trái. Hai vế phải tương thích về số cột và kiểu.

### Bản sao

Trong SQL Server, UNION loại dòng trùng và UNION ALL giữ dòng trùng; INTERSECT và EXCEPT trong handbook dùng dạng mặc định, không có biến thể ALL.

| Kiểm tra tính | Đáp án                                           |
|---------------|--------------------------------------------------|
| Ao6 EXCEPT    | P007 là ứng viên chưa bán trong seed.            |
| Ao6 INTERSECT | Kết quả phụ thuộc giao giữa hai miền mã độc lập. |

Các miền mã chỉ minh họa khả năng tương thích; không khẳng định hai mã là cùng một thực thể.

## 7. Gán nhãn kết quả mà không làm sai NULL

```
SELECT ProductId, ProductName,  
       CASE  
         WHEN Price >= 6000 THEN 'HIGH'  
         WHEN Price >= 1000 THEN 'MID'  
         ELSE 'LOW'  
       END AS PriceBand  
FROM dbo.tr_products;
```

CASE trả nhãn theo điều kiện; không biến NULL thành 0. Khi cần phân loại NULL, đặt nhánh `WHEN Price IS NULL` trước so sánh.

```
SELECT CustomerId,  
       CASE WHEN City IS NULL THEN 'UNKNOWN'  
         ELSE City END AS CityLabel  
FROM dbo.tr_customers;
```

### Checkpoint cuối lab

Tôi có thể chọn scalar, correlated, EXISTS, Double NOT EXISTS, set operator hay CASE từ cấu trúc câu hỏi.

A01–A07 được kiểm tra tĩnh; runtime phụ thuộc SQL Server cục bộ.

## 8. Bài tập nối tiếp

1. Tìm khách có đơn PAID nhưng không có dòng item; nêu vì sao LEFT JOIN phù hợp hơn EXISTS ở báo cáo này.
2. Tìm sinh viên có điểm cao nhất ở mỗi học phần; ghi cách xử lý dòng hạng.
3. Viết câu “mọi khóa học đều có ít nhất một lần thi” bằng Double NOT EXISTS, rõ ràng divisor rỗng.

| Tự kiểm    | Câu hỏi                                             |
|------------|-----------------------------------------------------|
| Miền       | Tôi đang xuất thực thể nào?                         |
| Liên kết   | Subquery tương quan bằng khóa nào?                  |
| Rỗng/trùng | Điều gì xảy ra khi divisor rỗng hoặc retake?        |
| Semantics  | Đang nói kết quả logic, không nói kết hoạch vật lý? |

### Bài nộp nhỏ

Nộp một trang trace: câu hỏi, miền ứng viên, điều kiện thiếu, truy vấn, expected key values và cách kiểm tra.

## LAB 04 · ANALYTICS

## Tổng hợp đúng nhóm, đọc đúng đồng hạng

Lab 04 dùng đơn hàng và kết quả học tập trong fixture để luyện COUNT, SUM, AVG, MIN, MAX, GROUP BY, HAVING, RANK và TOP WITH TIES. Expected values bên dưới được đổ i chiế u tính với seed.

### Mục tiêu

- Phân biệt WHERE và HAVING.
- Nhận diện lỗi Msg 8120.
- Giữ đồ ãng hạng đúng với RANK và TOP WITH TIES.

### Hai tầng suy luận

Lọc dòng trước khi nhóm bằ ãng WHERE; lọc nhóm sau khi tính aggregate bằ ãng HAVING.

Aggregate đối chiếu TECH-A02; RANK/TOP liên hệ LOC-SQL-LAB04; bài toán và tie cases là dữ liệu huấn luyện tự biên soạn.

### Tôi có thể...

gạch chân cột nhóm, cột đo lường và điề u kiện sau nhóm trước khi viế t SELECT.

## 1. Đếm và cộng theo đơn

```
SELECT o.OrderId,
       COUNT(*) AS LineCount,
       SUM(i.Qty * i.UnitPrice) AS OrderValue
FROM   dbo.tr_orders AS o
JOIN   dbo.tr_order_items AS i ON i.OrderId = o.OrderId
GROUP BY o.OrderId
ORDER BY o.OrderId;
```

| Biến     | Ý nghĩa                           | Kiểm chứng                                               |
|----------|-----------------------------------|----------------------------------------------------------|
| COUNT(*) | Số dòng item trong mỗi đơn.       | Không phải số sản phẩm khác nhau nếu trùng mã được phép. |
| SUM      | Tổng Qty × UnitPrice.             | DECIMAL giữ phần lẻ.                                     |
| GROUP BY | Một dòng kết quả cho mỗi OrderId. | Mọi cột không aggregate phải nằm trong GROUP BY.         |

### Thử mình

Đổi COUNT(\*) thành COUNT(DISTINCT ProductId) và giải thích hai câu hỏi khác nhau.

## 2. Lọc trước và sau nhóm

```
SELECT c.City, COUNT(*) AS CustomerCount
FROM dbo.tr_customers AS c
WHERE c.CustomerId <> 'C005'
GROUP BY c.City
HAVING COUNT(*) >= 2;
```

WHERE loại dòng trước khi GROUP BY; HAVING nhìn thấy aggregate của từng nhóm. Cột chọn không thuộc aggregate và không có trong GROUP BY gây Msg 8120.

```
-- SAI: c.FullName không nhóm và không aggregate
SELECT c.City, c.FullName, COUNT(*)
FROM dbo.tr_customers AS c
GROUP BY c.City;
```

### Cách sửa Msg 8120

Bỏ c.FullName, thêm c.FullName vào GROUP BY, hoặc chọn một aggregate phù hợp. Không “chừa” bằng DISTINCT nếu nó làm đổi câu hỏi.

### 3. Thống kê kết quả học tập

```
SELECT r.CourseId,
       AVG(r.Score) AS AvgScore,
       MIN(r.Score) AS MinScore,
       MAX(r.Score) AS MaxScore
FROM   dbo.tr_results AS r
GROUP BY r.CourseId
ORDER BY r.CourseId;
```

| Aggregate | NULL                                  | Retake                                 |
|-----------|---------------------------------------|----------------------------------------|
| AVG       | Bỏ qua NULL theo semantics aggregate. | Mỗi attempt là một dòng nếu không lọc. |
| MIN/MAX   | Không biến NULL thành 0.              | Độ nhúng không bị loại.                |

TRACE SUY LUẬN LOGIC

1

Chọn CourseId làm miền nhóm.

2

Tính ba aggregate trên các dòng thuộc nhóm.

3

Sắp xếp kết quả sau khi nhóm.

Nếu muốn “điểm cuối”, cần quy tắc chọn attempt trước; không tự gọi MAX là điểm cuối.

## 4. Xếp hạng trong từng học phần

```
SELECT r.CourseId, r.StudentId, r.Score,  
       RANK() OVER (  
         PARTITION BY r.CourseId  
         ORDER BY r.Score DESC  
       ) AS ScoreRank  
FROM dbo.tr_results AS r;
```

PARTITION BY tạo các nhóm xếp hạng độc lập; ORDER BY trong OVER quyết định thứ tự. RANK giữ cùng hạng và tạo khoảng trống sau tie (1, 1, 3).

| Hàm        | Ví dụ tie  | Hạng sau tie              |
|------------|------------|---------------------------|
| RANK       | 90, 90, 80 | 1, 1, 3                   |
| DENSE_RANK | 90, 90, 80 | 1, 1, 2                   |
| ROW_NUMBER | 90, 90, 80 | 1, 2, 3; cần tie-breaker. |

### Thử mình

Thêm StudentId vào ORDER BY của ROW\_NUMBER rồi nêu vì sao kết quả khác RANK.

## 5. Lấy ngưỡng cao nhất mà không cắt đồng hạng

```
SELECT TOP (2) WITH TIES
       ProductId, ProductName, Price
FROM   dbo.tr_products
ORDER BY Price DESC;
```

TOP (2) WITH TIES có thể trả hơn hai dòng nếu dòng kế tiếp bằng giá trị ORDER BY ở ranh giới. Không đặt ORDER BY thì TOP không có thứ tự để diễn giải.

| Câu lệnh          | Ý nghĩa                                                 |
|-------------------|---------------------------------------------------------|
| TOP (2)           | Giới hạn hai dòng sau thứ tự đã nêu.                    |
| TOP (2) WITH TIES | Giữ tất cả dòng đồng hạng ở ngưỡng thứ hai.             |
| RANK() <= 2       | Cùng ý niệm hạng, nhưng trả theo từng partition nếu có. |

### Sai thường gặp

TOP WITH TIES xét toàn bộ biểu thức ORDER BY. Nếu thêm ProductId vào ORDER BY, tie theo Price có thể bị phá.

## 6. Lọc đơn có tổng giá trị

```
SELECT o.CustomerId,  
       COUNT(DISTINCT o.OrderId) AS OrderCount,  
       SUM(i.Qty * i.UnitPrice) AS TotalValue  
FROM   dbo.tr_orders AS o  
JOIN   dbo.tr_order_items AS i ON i.OrderId = o.OrderId  
GROUP BY o.CustomerId  
HAVING SUM(i.Qty * i.UnitPrice) > 500000;
```

COUNT DISTINCT để m đơn, còn SUM cộng giá trị dòng. Nếu u join thêm bảng có quan hệ 1-n nữa, phải kiểm soát việc nhân dòng trước khi cộng.

### TRACE SUY LUẬN LOGIC

- 1 Join orders với items theo OrderId.
- 2 Nhóm theo CustomerId.
- 3 HAVING giữ nhóm có TotalValue vượt ngưỡng.

### Checkpoint

Tôi có thể giải thích vì sao điều kiện SUM nằm ở HAVING, không nằm ở WHERE.

## 7. Bài tập có tie và NULL

1. Thêm một sản phẩm có cùng Price cao nhất trong transaction, chạy TOP (1) WITH TIES và ghi số dòng.
2. Tìm mỗi khách có tổng giá trị lớn nhất; so sánh RANK với TOP.
3. Tính AVG Score theo học phần sau khi loại attempt không hợp lệ bằng WHERE.

| Tự kiểm  | Đáp án phải nêu                        |
|----------|----------------------------------------|
| Nhóm     | Cột nào quyết định một group?          |
| NULL     | Aggregate xử lý NULL ra sao?           |
| Tie      | TOP, RANK, ROW_NUMBER khác nhau ở đâu? |
| Expected | Đã đổi chiều seed hay đang minh họa?   |

Tie case được tạo riêng trong bài tập; không sửa seed chuẩn nếu cần lặp lại các expected rows A01–A07.

## 8. Bảng quyết định nhanh

| Nếu đề hỏi             | Bắt đầu bằng           | Kiểm tra                    |
|------------------------|------------------------|-----------------------------|
| Số / tổng / trung bình | GROUP BY + aggregate   | NULL, retake, nhân dòng     |
| Nhóm đạt ngưỡng        | HAVING                 | Không đặt aggregate ở WHERE |
| Top toàn cục có tie    | TOP WITH TIES          | ORDER BY và biểu thức tie   |
| Hạng theo nhóm         | RANK OVER PARTITION BY | Khoảng hạng sau tie         |

### Tách câu hỏi

Viết câu SELECT không aggregate trước, xác nhận miền dòng, rồi mới thêm grouping và điều kiện sau nhóm.

Lab 04 hoàn tất phần analytics; chuyển sang debugging để học cách đọc lỗi thay vì sửa ngẫu nhiên.

## PART 11 · DEBUGGING

## Từ thông báo lỗi đến giả thuyết kiểm chứng

Debug thực hành bắt đầu bằng câu hỏi: lỗi ở ngữ cảnh, tên đối tượng, kiểu dữ liệu, ràng buộc hay logic? Mỗi thẻ bên dưới có symptom, cause, check, fix và prevent.

### QUY TRÌNH 5 BƯỚC

- 1 Đọc mã Msg và dòng lỗi.
- 2 Xác nhận DB\_NAME(), schema, alias.
- 3 Thu hẹp câu lệnh thành ví dụ nhỏ.
- 4 Sửa một nguyên nhân, chạy lại.
- 5 Ghi test ngăn tái diễn.

Mã lỗi là bảng chẩn đoán thực hành; cú pháp trigger đối chiếu TECH-A04; ví dụ là fixture nội bộ, không phải bản sao bài làm.

### Tôi có thể...

viết giả thuyết trước khi thay đổi câu SQL.

## 1. Không tìm thấy object hoặc column

### Msg 208

**SYMPTOM**Invalid object name.

**CAUSE**Đang ở nhầm database, sai schema hoặc chưa chạy DDL.

**CHECK** `SELECT DB_NAME(); SELECT OBJECT_ID(N'dbo.tr_orders');`

**FIX**Chạy `USE IT004_Training`; tạo schema đúng thứ tự.

**PREVENT**Đặt dòng kiểm tra ngữ cảnh ở đầu script.

### Msg 207

**SYMPTOM**Invalid column name.

**CAUSE**Sai chính tả, alias hoặc dùng tên cột của bảng khác.

**CHECK**Đọc `sys.columns` và qualify mọi cột khi join.

**FIX**Sửa tên/alias, không thêm cột đoán mò.

**PREVENT**Ghi lược đồ cạnh câu query.

### Kiểm tra nhỏ

Chạy `SELECT TOP (1) * FROM dbo.tr_orders` trước khi viết join dài.

## 2. FK, CHECK và khóa trùng

### Msg 547

**SYMPTOM**INSERT/UPDATE/DELETE xung đột constraint.

**CAUSE**FK không có cha, CHECK sai hoặc xóa dòng còn được tham chiếu.

**CHECK**Đọc tên FK/CK trong thông báo; SELECT dòng cha/con.

**FIX**Sửa thứ tự/nội dung; không tắt constraint để che lỗi.

**PREVENT**Seed parent trước child; test mỗi constraint.

### Msg 2627 / 2601

**SYMPTOM**Violation PRIMARY KEY hoặc UNIQUE index.

**CAUSE**Mã/cặp khóa đã tồn tại.

**CHECK**SELECT theo đúng khóa, kể cả khóa ghép.

**FIX**Đổi mã hoặc cập nhật có chủ đích.

**PREVENT**Idempotent seed và transaction rollback.

Trong fixture, (OrderId, ProductId) là khóa ghép; chỉ kiểm tra một thành phần là chưa đủ.

### 3. Tràn số và cắt chuỗi

**Msg 8115**

**SYMPTOM**Arithmetic overflow.

**CAUSE**Giá trị vượt precision/scale hoặc chuyển kiểu số quá hẹp.

**CHECK**Kiểm tra DECIMAL(p,s), kiểu trung gian và phép nhân.

**FIX**Chọn precision phù hợp, CAST có chủ đích.

**PREVENT**Test biên và không ép kiểu âm thậ̀m.

**Msg 8152 / 2628**

**SYMPTOM**String or binary data would be truncated.

**CAUSE**Chuỗi dài hơn cột đích.

**CHECK**DATALENGTH và metadata cột.

**FIX**Sửa dữ liệu hoặc thiết kế độ dài; không cắt tùy tiện.

**PREVENT**Kiểm tra Unicode và độ dài trước INSERT.

```
SELECT DATALENGTH(N'chuỗi thử') AS BytesUsed;
```

## 4. Nhóm sai và scalar nhiều dòng

### Msg 8120

**SYMPTOM**Cột SELECT không hợp lệ vì không nằm trong GROUP BY.

**CAUSE**Trộn cột chỉ tiết với aggregate.

**CHECK**Liệt kê từng cột SELECT.

**FIX**Thêm vào GROUP BY hoặc aggregate đúng câu hỏi.

**PREVENT**Vẽ grain của kết quả trước query.

### Msg 512

**SYMPTOM**Subquery trả nhiều hơn một giá trị.

**CAUSE**Dùng `= (subquery)` khi kết quả không scalar.

**CHECK**Chạy subquery độc lập và COUNT(\*)

**FIX**IN/EXISTS hoặc aggregate.

**PREVENT**Ghi cardinality mong đợi cạnh subquery.

### Thử mình

Sửa query GROUP BY trong Lab 04 mà không dùng DISTINCT như miêu tả.

## 5. Kiểu dữ liệu, ngày và NULL

### Msg 245

**SYMPTOM**Conversion failed when converting varchar to int.

**CAUSE**Literal hoặc cột chứa dữ liệu không chuyển được.

**CHECK**TRY\_CONVERT trên dữ liệu nghi ngờ.

**FIX**Chuẩn hóa kiểu và literal, không dựa vào precedence mở hô`.

**PREVENT**Dùng kiểu đúng ngay ở schema.

### Msg 241 / 242

**SYMPTOM**Conversion failed hoặc out-of-range date/time.

**CAUSE**Chuỗi ngày mở hô` hoặc giá trị ngoài miê`n.

**CHECK**Dùng ISO 8601, TRY\_CONVERT(datetime2,...).

**FIX**Sửa literal và khoảng nửa kín.

**PREVENT**Không dùng format phụ thuộc ngôn ngữ.

NULL câ`n `IS NULL` / `IS NOT NULL` ; mọi phép so sánh thông thường với NULL cho UNKNOWN.

## 6. Ambiguous, invalid và “tất cả” sai

| Symptom              | Cause                                   | Check / fix                        |
|----------------------|-----------------------------------------|------------------------------------|
| Ambiguous column     | Hai bảng cùng tên cột, không qualify.   | Thêm alias và tiền tố bảng.        |
| Invalid column/alias | Alias chưa tồn tại ở scope hiện tại.    | Đọc scope SELECT, FROM, subquery.  |
| Universal trả thừa   | COUNT(*) để m retake hoặc divisor rỗng. | COUNT DISTINCT, nêu empty-divisor. |
| NOT EXISTS trả thiếu | Chọn miề n ứng viên từ bảng con.        | Bắ t đầ u từ thực thể cầ n xuấ t.  |

```
SELECT s.StudentId
FROM dbo.tr_students AS s
WHERE NOT EXISTS (SELECT 1 FROM dbo.tr_courses AS c WHERE NOT EXISTS (SELECT 1 FROM dbo.tr_results AS
r WHERE r.StudentId=s.StudentId AND r.CourseId=c.CourseId));
```

### Checkpoint

Tôi có thể chỉ ra miề n ứng viên và dòng thiế u trước khi sửa Double NOT EXISTS.

## 7. Debug trigger theo event

Một trigger statement-level nhận tập inserted/deleted. Không đọc một biến scalar từ bảng ảo rồi giả định chỉ có một dòng.

| Event     | Câu hỏi                    | Check                                            |
|-----------|----------------------------|--------------------------------------------------|
| INSERT    | Dòng mới có cha hợp lệ?    | inserted anti-join parent                        |
| UPDATE    | Khóa nào đổi?              | JOIN inserted/deleted theo PK                    |
| DELETE    | Dòng bị xóa còn là head?   | deleted anti-join current employees              |
| Multi-row | Có một dòng vi phạm không? | EXISTS trên tập vi phạm; rollback toàn statement |

Đổi chiều `practice/sql/05_triggers.sql` và các ca A–H: A/B/C/G pass; D/F/H reject theo trigger; riêng ca E bị schema-level reject vì `DeptId` là `NOT NULL`, trước khi trigger có thể chạy.

### Failure mode scalar

Pattern `SELECT @id = EmployeeId FROM inserted` chỉ tình cờ đúng cho một dòng và bỏ qua vi phạm trong dòng khác.

Trigger semantics đối chiếu TECH-A04, TECH-A05 và TECH-A06; event table là bài biên soạn riêng.

## 8. Nhật ký debug

| Thời điểm | Msg / symptom | Giả thuyết | Test | Kết quả |
|-----------|---------------|------------|------|---------|
|           |               |            |      |         |
|           |               |            |      |         |

1. Chọn một lỗi từ Lab 02 và điền đủ năm cột.
2. Viết một test prevent, ví dụ để m dòng hoặc kiểm tra FK.
3. Chạy lại trên database reset, không dùng trạng thái ngẫu nhiên.

### Tôi có thể...

biến mỗi lỗi thành một kiểm tra lặp lại được, thay vì chỉ sửa đến khi câu lệnh chạy.

## PART 12 - PRACTICAL EXAM

## Quy trình giải một bài thực hành

Trong snapshot Phase A, các artifact thực hành thường xoay quanh DDL, nạp dữ liệu, truy vấn, ràng buộc và kiểm tra lỗi. Đây là bản đồ chuẩn bị, không phải tuyên bố rằng mọi kỳ thi đều giống nhau.

### 10 ARTIFACT CẦN CHUẨN BỊ

- 1 Tạo database và chọn context.
- 2 DDL: bảng, kiểu, PK/FK/CHECK.
- 3 Nạp parent rồi child.
- 4 SELECT lọc, sắp xếp, NULL/date.
- 5 JOIN và self join.
- 6 Aggregate/GROUP/HAVING.
- 7 Subquery/EXISTS/universal.
- 8 Set operators/CASE/RANK.
- 9 RBTV/trigger và test nhiễu dòng.
- 10 Đóng gói script, output và nhật ký lỗi.

Provenance: PRAC-2024-2025-HK1-302, PRAC-2024-2025-HK1-01, PRAC-2023-2024-HK1-FINAL-01, PRAC-2022-2023-HK1-D03 và exam-pattern-map của Phase A; nhãn "snapshot" giữ đúng mức độ chắc chắn của nguồn.

## 1. Phân bổ thời gian tham khảo

| Khoảng   | Việc nên làm                              | Điểm dừng                |
|----------|-------------------------------------------|--------------------------|
| 10% đầu  | Đọc lược đồ, đánh dấu PK/FK, tạo context. | DB_NAME đúng.            |
| 25%      | DDL và dữ liệu mẫu nhỏ.                   | SELECT kiểm tra số dòng. |
| 35%      | Query từ dễ đến khó; ghi expected key.    | Không còn alias mơ hồ.   |
| 20%      | RBTV/trigger và test A–H.                 | Multi-row đã thử.        |
| 10% cuối | Reset, chạy lại, đóng gói.                | Không còn file tạm.      |

Phân bổ chỉ là **THAM KHẢO**; điều chỉnh theo đề và môi trường thực tế. Không tự suy ra quy định tên file hoặc thang điểm nếu đề không nêu.

**Bàn giao**

Mỗi script có comment mục đích, expected behavior và một test tối thiểu. Giữ nguyên thứ tự chạy để người khác tái lập.

## 2. Cú pháp cần nhớ

| Nhu cầu       | Mẫu                                        |
|---------------|--------------------------------------------|
| Ngữ cảnh      | USE IT004_Training;                        |
| NULL          | col IS NULL , col IS NOT NULL              |
| Ngày          | col >= @d AND col < @next                  |
| Có / không có | EXISTS / NOT EXISTS                        |
| Tất cả        | Double NOT EXISTS hoặc COUNT DISTINCT      |
| Nhóm          | GROUP BY , aggregate, HAVING               |
| Đánh hạng     | RANK() OVER(PARTITION BY ... ORDER BY ...) |
| Top có tie    | TOP (n) WITH TIES ... ORDER BY ...         |

```
SELECT TOP (5) WITH TIES ProductId, Price
FROM dbo.tr_products
ORDER BY Price DESC;
```

### Tôi có thể...

chọn đúng mẫu mà không sao chép toàn bộ query cũ.

### 3. Lỗi theo nguyên nhân

| Nhóm       | Mã / dấu hiệu       | Phản xạ đầu tiên                        |
|------------|---------------------|-----------------------------------------|
| Context    | Wrong DB, Msg 208   | DB_NAME(), OBJECT_ID                    |
| Name       | Msg 207, ambiguous  | sys.columns, qualify alias              |
| Constraint | Msg 547             | Đọc tên FK/CK, kiểm tra cha/con         |
| Duplicate  | Msg 2627/2601       | Tìm đúng khóa, kể cả khóa ghép          |
| Size       | Msg 8115, 8152/2628 | Kiểm tra kiểu và DATALENGTH             |
| Conversion | Msg 245, 241/242    | TRY_CONVERT, ISO date                   |
| Logic      | Msg 8120, 512       | Grain/group và cardinality subquery     |
| NULL       | Kết quả UNKNOWN     | IS NULL, COALESCE có chủ đích           |
| Universal  | Thiếu thừa dòng     | Miền ứng viên, divisor rỗng, duplicates |

Bảng là bản đồ chẩn đoán; mã Msg có thể đi kèm văn bản cụ thể tùy phiên bản SQL Server.

### 4. Bản đồ nguồn và cách dùng

| Mã                                                                                                 | Vai trò                    | Đường dẫn / URL                                              | Cách dùng                                        |
|----------------------------------------------------------------------------------------------------|----------------------------|--------------------------------------------------------------|--------------------------------------------------|
| TECH-Ao1...TECH-A11                                                                                | Microsoft technical        | research/v1.1_phase_a/source_inventory.md (exact URL per ID) | Semantics T-SQL; đổi i chiế u URL trong Phase A. |
| LOC-SQL-LABo1...04                                                                                 | Local observed progression | research/v1.1_phase_a/                                       | Định hướng thứ tự lab.                           |
| PRAC-2024-2025-HK1-302, PRAC-2024-2025-HK1-01, PRAC-2023-2024-HK1-FINAL-01, PRAC-2022-2023-HK1-D03 | Practical relevance        | research/v1.1_phase_a/exam_pattern_map.md                    | Chỉ nói “snapshot” khi mô tả pattern.            |
| UIT official                                                                                       | Course context             | research/v1.1_phase_a/source_inventory.md (exact URL)        | Bô i cảnh môn học, không lấ y làm semantics SQL. |

Student/community mirrors chỉ là nguồ n khám phá và provenance; không trích nguyên văn, không dùng làm thẩm quyề n kỹ thuật. Mỗi nguồ n web đầ y đủ exact URL trong hồ sơ Phase A.

**Ba nhãn độ tin cậy**  
VERIFIED = nguồ n trực tiế p; RECONSTRUCTED = suy ra từ nhiề u dấ u vể t; INFERENCE = nhận định biên soạn.  
Không đổi nhâ n để làm câu chuyện chấ c hơn.

## 5. Tính năng ngoài lõi

| Tính năng             | Trạng thái          | Ghi chú                                                 |
|-----------------------|---------------------|---------------------------------------------------------|
| Stored procedure      | Tùy chọn nâng cao   | Chỉ thêm sau khi query lõi ổn định.                     |
| View                  | Tùy chọn            | Dùng để đóng gói SELECT, không che semantics.           |
| UDF                   | Lịch sử / không lỗi | Không đưa vào bài bắt buộc nếu môi trường không hỗ trợ. |
| Transaction / locking | Sandbox             | Không coi là pattern exam đã xác minh.                  |

Scope thực hành ưu tiên DDL, DML, SELECT, integrity và debugging. Mở rộng phải giữ dataset và expected result nhất quán.

### Thử mình

Chuyển một query Bo3 thành view tùy chọn, rồi kiểm tra view vẫn trả cùng khóa và số dòng.

6. Phiếu đối chiếu kết quả

| Example ID | Script / dataset              | Expected key value                       | Mode         |
|------------|-------------------------------|------------------------------------------|--------------|
| B01        | o2_seed + o3_queries_basic    | 3 dòng: Coo1, Coo2, Coo3                 | STATIC       |
| B02        | o2_seed + o3_queries_basic    | OrderDate trong [2024-10-15, 2024-10-18) | STATIC       |
| B03        | o2_seed + o3_queries_basic    | 11 dòng join                             | STATIC       |
| B04        | o2_seed + o3_queries_basic    | 5 dòng; Coo5 có OrderCount 0             | STATIC       |
| B05        | o2_seed + o3_queries_basic    | 4 dòng: Eoo1–Eoo4                        | STATIC       |
| A02        | o2_seed + o4_queries_advanced | 3 dòng: Poo1, Poo3, Poo5                 | STATIC       |
| A03        | o2_seed + o4_queries_advanced | 1 dòng: Poo7 chưa có item                | STATIC       |
| A04        | o2_seed + o4_queries_advanced | 1 dòng: Soo1; empty divisor đã nêu       | STATIC       |
| A05        | o2_seed + o4_queries_advanced | 1 dòng: Soo1                             | STATIC       |
| A06        | o2_seed + o4_queries_advanced | INTERSECT 0; EXCEPT Poo7                 | STATIC       |
| A07        | o2_seed + o4_queries_advanced | 7 dòng; Poo1/Poo4/Poo7 LOW               | STATIC       |
| TOP-TIE    | Lab 04 tie exercise           | Có thể hơn n dòng khi tie                | ILLUSTRATIVE |

Runtime validation chỉ được đánh dấu RUNTIME khi SQL Server thực sự chạy và log được output. Các hàng còn lại là STATIC hoặc ILLUSTRATIVE, không giả làm kết quả thực thi.

Quy tắc cập nhật

Nếu sửa seed, sửa expected key values và report cùng commit.

## 7. Trước khi nộp bài

- Database/context đúng; script chạy theo thứ tự.
- PK, FK, CHECK và circular FK đã được kiểm tra.
- Query có alias, NULL/date đúng và không dựa vào thứ tự vật lý.
- GROUP BY/HAVING, RANK/TOP WITH TIES phân biệt rõ.
- Trigger đã thử inserted/deleted nhiều dòng và ca A–H.
- Expected rows ghi đúng mode STATIC/RUNTIME/ILLUSTRATIVE.
- Không có file nguồn có bản quyền hoặc file tạm.

### Tôi có thể...

reset fixture, chạy lại từ đầu và giải thích mỗi kết quả then chốt.

Checklist biên soạn cho handbook này; không phải biểu mẫu chấm điểm của UIT.

## 8. Lời kết

Cuốn sổ tay thực hành này kết thúc ở một quy trình có thể lặp lại: đọc lược đồ, viết giả thuyết, chạy câu lệnh nhỏ, kiểm tra kết quả và ghi lại nguyên nhân lỗi.

schema → query → result → test

Fixture `IT004_Training`, các script SQL và report QA đi cùng tài liệu để người học có thể tự reset và kiểm chứng. Những phần chưa được xác minh runtime được ghi rõ thay vì che bằng con số.

### Giữ tính trung thực

Nguồn được phân tâm; câu hỏi exam chỉ được gọi là snapshot khi có provenance tương ứng. Nội dung kỹ thuật tách khỏi nguồn khám phá.

Biên soạn: Võ Trọng Phúc · IT004 — Thực hành Cơ sở dữ liệu